

CyberQuest

Synchron bewegt, doppelt gesiegt

Praktikumsbericht von Artjom Becker, Ali Cebeci, Philip Fauss und Ruben Gielisch

Tag der Einreichung: 10. September 2025

Gutachter: Dr.-Ing. (habil.) Stefan Göbel

Betreuer: Florian Horn

Darmstadt



TECHNISCHE
UNIVERSITÄT
DARMSTADT

et:t

Fachbereich
Elektrotechnik und
Informationstechnik



AG Serious
Games

Erklärung zur Abschlussarbeit gemäß § 22 Abs. 7 APB TU Darmstadt

Hiermit erkläre ich, Artjom Becker, Ali Cebeci, Philip Fauss und Ruben Gielisch, dass ich die vorliegende Arbeit gemäß § 22 Abs. 7 APB der TU Darmstadt selbstständig, ohne Hilfe Dritter und nur mit den angegebenen Quellen und Hilfsmitteln angefertigt habe. Ich habe mit Ausnahme der zitierten Literatur und anderer in der Arbeit genannter Quellen keine fremden Hilfsmittel benutzt. Die von mir bei der Anfertigung dieser wissenschaftlichen Arbeit wörtlich oder inhaltlich benutzte Literatur und alle anderen Quellen habe ich im Text deutlich gekennzeichnet und gesondert aufgeführt. Dies gilt auch für Quellen oder Hilfsmittel aus dem Internet.

Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Mir ist bekannt, dass im Falle eines Plagiats (§ 38 Abs. 2 APB) ein Täuschungsversuch vorliegt, der dazu führt, dass die Arbeit mit 5,0 bewertet und damit ein Prüfungsversuch verbraucht wird. Abschlussarbeiten dürfen nur einmal wiederholt werden.

Bei einer Thesis des Fachbereichs Architektur entspricht die eingereichte elektronische Fassung dem vorgestellten Modell und den vorgelegten Plänen.

Darmstadt, 10. September 2025



A. Becker, A. Cebeci,
P. Fauss, R. Gielisch

Inhaltsverzeichnis

1. Projektdefinition	5
2. Projektplanung	7
2.1. Komponentendiagramm und Projektplan	7
2.1.1. Gantt-Diagramm	9
3. Konzeption und prototypische Umsetzung	12
3.1. Beatmaps und deren Erstellung	12
3.1.1. Konzept	12
3.1.2. Prototyp	12
3.1.3. Methodik	14
3.2. Rhythmusspiel	15
3.2.1. Konzept	15
3.2.2. Prototyp	16
3.2.3. Methodik	17
3.2.4. Evaluation	17
3.3. Metaspiel	19
3.3.1. Konzept	19
3.3.2. Prototyp	20
3.3.3. Methodik	21
4. Zusammenfassung und Ausblick	23
4.1. Zusammenfassung	23
4.1.1. Beatmap Creator	23
4.1.2. Spiel	23
4.1.3. Methodik	24
4.2. Lessons Learned	25
4.3. Ausblick	25
A. Anhang	27
A.1. Bedienungsanleitung	27
A.1.1. Vorbereitung	27
A.1.2. Hauptmenü	27
A.1.3. Das Spiel	28
A.1.4. Beatmap-Creator	28
A.2. Technisches Manual	29
A.2.1. Beatmap-Creator	29
A.2.2. Rhythmusspiel	30
A.2.3. Metaspiel	33

A.2.4. Tracking	36
A.3. Verwendete Technologien	36
A.4. Erweiterung des Game Experience Questionnaire (GEQ)	37
A.4.1. Beobachtungspunkte	37
A.4.2. Körperliche Aktivität und Spielerlebnis	37
A.4.3. Motivation	37
A.4.4. Visuelle Verständlichkeit	38
A.4.5. Handtrennung	38
A.4.6. Tracking	38
A.5. Weitere Abbildungen	38
A.6. Protokolle	56

1. Projektdefinition

Thema und Herausforderungen: Rythmus basiertes kooperatives Exergame mit einem Roguelike Metagame. Spieler:innen bewegen sich synchron zu Musik und übernehmen in der futuristischen Cyberwelt Aufgaben und Kämpfe.

Hierbei stellen sich verschiedene Herausforderung, vor allem auf der technischen Seite. Da das Spiel in Echtzeit abläuft und gute Koordination und Timing erfordert, ist es notwendig, dass den Spielenden zu jedem Zeitpunkt klar ist, was sie wo tun müssen. Diese Visual Clarity zu erzeugen ist eine der Hauptherausforderungen. Eine weitere Herausforderung, die eine ähnliche Problematik behandelt, ist es den Spielerinput richtig zu interpretieren, also das Timing so einzustellen, dass ein gutes Spielerlebnis erschaffen wird.

Die gute Integration von kooperativen Elementen, die Zusammenarbeit fördert, ohne künstlich zu wirken ist nicht zu unterschätzen.

Eine weitere Herausforderung ist das Erstellen von Beatmaps¹, bei dem darauf geachtet werden muss, dass sie einen guten Herausforderungsgrad enthalten, um die Spielenden im Flow zu halten.

Zu guter Letzt ist das Roguelike Metagame eine große Herausforderung, da es quasi ein weiteres Spiel in dem Spiel darstellt, welches nahtlos in das Spielerlebnis integriert werden muss.

Zielgruppe: Eine Zielgruppenbeschränkung gibt es für dieses Spiel per se nicht. Fitnessbegeisterte jedes Alters sollen von diesem Spiel angesprochen werden.

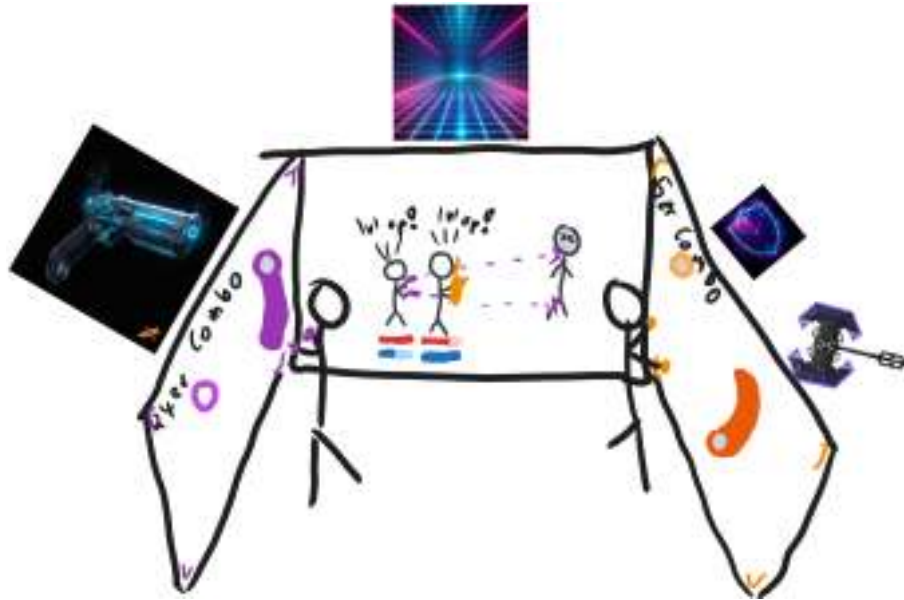
Mehrwert: Die Zielgruppe profitiert hauptsächlich davon, dass das mühsame kardiovaskuläre Training durch Gamification spaßiger gestaltet wird und somit die Nutzenden motiviert sich mehr zu bewegen. Ein weiterer Aspekt, welches das Spiel behandelt, sind soziale Kompetenzen, die im kooperativen Modus entwickelt und gestärkt werden. Die Motivation fürs Spielen wird ebenfalls durch die Zusammenarbeit im Spiel zusätzlich gefördert.

Inhalte: Für die Umsetzung des Projektes sind folgende Technologien vorgesehen:

- Unity
- Exercube
- Sphery SDK
- HTC Vive Tracker (3.0)
- Eigene 3D Umgebung, Charakter bzw. Avatar Design
- Eigene oder Lizenz freie Musik und Soundeffekte

¹Eine Beatmap ist eine formale Beschreibung wann im zu spielenden Lied, welche Bewegung ausgeführt werden muss.

Ergebnis: Das Ergebnis unseres Projekts ist ein funktionaler Prototyp des Exergames CyberQuest, der erfolgreich in den ExerCube integriert wurde. Die Steuerung der Charaktere erfolgt durch Bewegung innerhalb des ExerCube. Der Schwierigkeitsgrad sollte dabei variable an die körperlichen, sowie den mentalen Fähigkeiten angepasst, um eine progressive Verbesserung der Fähigkeiten zu gewährleisten.



2. Projektplanung

2.1. Komponentendiagramm und Projektplan

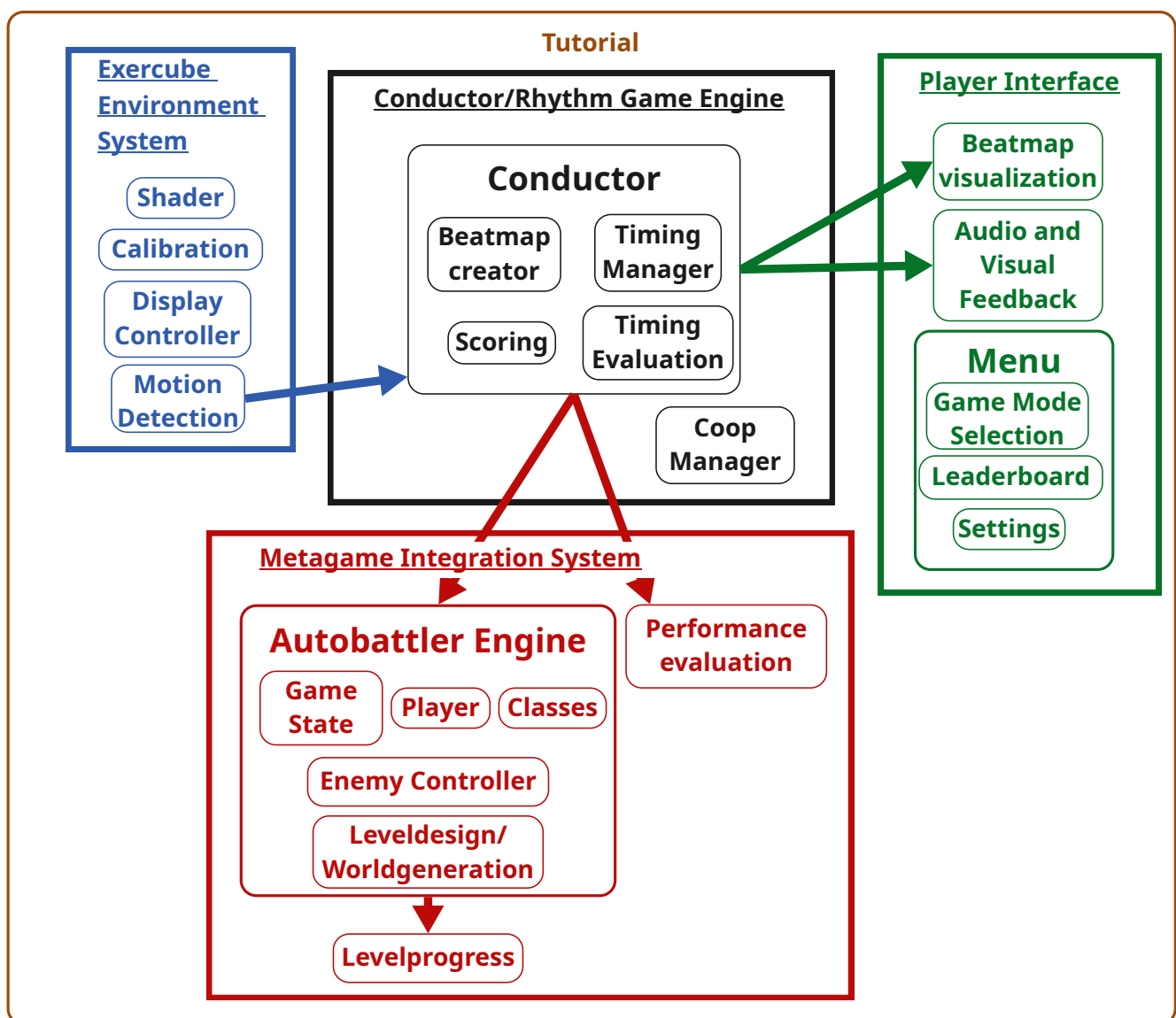


Abbildung 2.1.: Komponentendiagramm

Im Rahmen der Projektplanung wurden vier zentrale Entwicklungsbereiche (Abbildung 2.1) identifiziert, die im Verlauf der Veranstaltung systematisch bearbeitet werden müssen. Zusätzlich wurde eine übergreifende Komponente definiert, welche alle Hauptsysteme zusammenführt: das Tutorial. Im Folgenden werden die vier Kernbereiche näher erläutert.

Der erste Bereich betrifft das sogenannte **Exercube Environment System**, also das technische Fundament des gesamten Projekts. Der Exercube stellt die Hardwareplattform dar, auf der das Spiel letztlich ausgeführt wird. Um die technischen Möglichkeiten, aber auch die Limitierungen dieses Systems einschätzen zu können, ist eine grundlegende Einarbeitung aller Teammitglieder erforderlich. Darüber hinaus wird eine vertiefte Auseinandersetzung einzelner Gruppenmitglieder mit spezifischen Aspekten notwendig. Insbesondere gilt es, die Ausgabesignale des Exercube - vor allem die der VR-Controller - sinnvoll mit den Spielmechaniken zu verknüpfen und eine korrekte Projektion der Spielausgabe auf die Exercubewände zu gewährleisten. Lead Entwickler dieses Bereiches ist Artjom Becker (AB).

Ein zweiter zentraler Bereich umfasst das **Player Interface**, also die Schnittstelle zwischen Mensch und Maschine. Besonders bei Rhythmusspielen, die von präzisiertem Timing und unmittelbarem, hochwertigem Feedback leben, spielt die Qualität dieser Schnittstelle eine wichtige Rolle. Neben allgemeinen Meta-Einstellungen wie der Anpassung von Lautstärke oder Liedverzögerung¹ liegt ein Fokus auf der Gestaltung der Anzeige der Spielanweisungen. Beatmaps müssen in klar verständliche, visuelle Hinweise übersetzt werden, um eine motivierende Spielerfahrung zu ermöglichen. Lead Entwickler dieses Bereiches ist Ali Cebeci (AC).

Der dritte Bereich betrifft die **Conductor- bzw. Rhythm Game Engine**. Diese Komponente bildet das Herzstück des Spiels, da sie sämtliche zeitkritischen Abläufe koordiniert. Sie ist damit genauso wichtig wie der zweite Bereich. Der sogenannte Timing Manager stellt sicher, dass sämtliche Rhythmuskomponenten präzise synchronisiert agieren. Die Quelle der hierfür notwendig Daten bilden die Beatmaps, die mittels eines eigens dafür entwickelten Beatmap Creator erschaffen werden. Ergänzt wird das System durch ein Punktesystem, das einerseits den Spielenden unmittelbares Feedback gibt und andererseits als Datenquelle für weitere Systeme, wie beispielsweise das Metagame, dient. Lead Entwickler dieses Bereiches ist Philip Fauss (PF).

Der vierte Bereich, das **Metagame Integration System**, ergänzt die eigentliche Spielmechanik um eine motivationale Metaebene. Hier steht ein mit dem Punktesystem gekoppelter, modular entwickelter Autobattler im Zentrum. Ziel dieses Systems ist es, den Spielenden über das Rhythmusspiel hinaus eine langfristige Progression zu ermöglichen. Dabei werden Spiellevel, Gegnerdesigns und potenziell unterschiedliche Charaktertypen entwickelt, die dynamisch auf die im Rhythmusspiel erzielten Leistungen reagieren. Durch diese Verknüpfung entsteht ein zusätzlicher Anreiz zur spielerischen Verbesserung, ohne das Rhythmuserlebnis selbst zu überfrachten. Das Metagame wird bewusst modular entwickelt, um es in der Zukunft einfach austauschbar zu machen. Lead Entwickler dieses Bereiches ist Ruben Gielisch (RG).

Ergänzt werden diese vier Hauptbereiche durch eine eigenständige Tutorial-Komponente, die als Einstieg in das Spielgeschehen dient. Aufgrund des übergreifenden Charakters wird es keiner spezifischen Hauptkomponente zugeordnet, sondern als übergreifende Einheit betrachtet, die zur Kohärenz des gesamten Spielerlebnisses beiträgt.

In Abbildung 2.2 ist die voraussichtliche Zeitplanung dargestellt. Bei der Planung wurde darauf geachtet, dass als wichtig eingeschätzte Komponenten proportional mehr Zeit erhielten, als andere. Des Weiteren wurde die für das Erlernen von Unity notwendige Zeit in den einzelnen Komponenten berücksichtigt.

¹Um die Spielelemente besser mit dem Lied des Spieles zu synchronisieren ist es notwendig für verschiedene Geräte und Umgebungen eine variable Einstellung einer Verzögerung bereitzustellen.

Abbildung 2.2.: Projektplan

2.1.1. Gantt-Diagramm

Die Entwicklung des Projekts beginnt am 13. Mai 2025 und verläuft in mehreren klar definierten Phasen, die im Laufe der Zeit dynamisch ineinandergreifen. In der Frühphase des Projekts liegt der Schwerpunkt zunächst auf der Aneignung technischer Grundlagen. Diese dient als Voraussetzung dafür, um zügig in die Implementierung der zentralen Spielmechaniken übergehen zu können. Während der Anfangsphase entsteht eine initiale Version des Rhythmusspiels sowie des Player Interfaces mit dem Ziel, frühzeitig einen funktionalen Prototypen zu realisieren, der als Grundlage für den Alpha-Meilenstein dient.

Mit zunehmendem Projektfortschritt verschiebt sich der Fokus auf die Entwicklung des modular aufgebauten Metaspiels. Diese zweite Phase ist stark von den in der Rhythmuskomponente etablierten Strukturen abhängig und erweitert das bestehende Spielerlebnis um eine motivationale Ebene. Der konkrete zeitliche Ablauf sowie die zugehörigen Abhängigkeiten lassen sich dem Gantt-Diagramm in Abbildung 2.3 entnehmen.

Zu Beginn wurden mehrere zentrale Meilensteine definiert, die eng mit den jeweiligen Projektphasen verzahnt sind. Der erste dieser Meilensteine, **Exercube**, widmet sich dem Verständnis der technischen Grundlage. Ziel ist es, die Funktionsweise der Bewegungs- und Interaktionserkennung mithilfe der Exercube-SDK zu verstehen und eine geeignete Integration zu erarbeiten, über die das Rhythmusspiel sinnvoll mit der Hardware verknüpft werden kann.

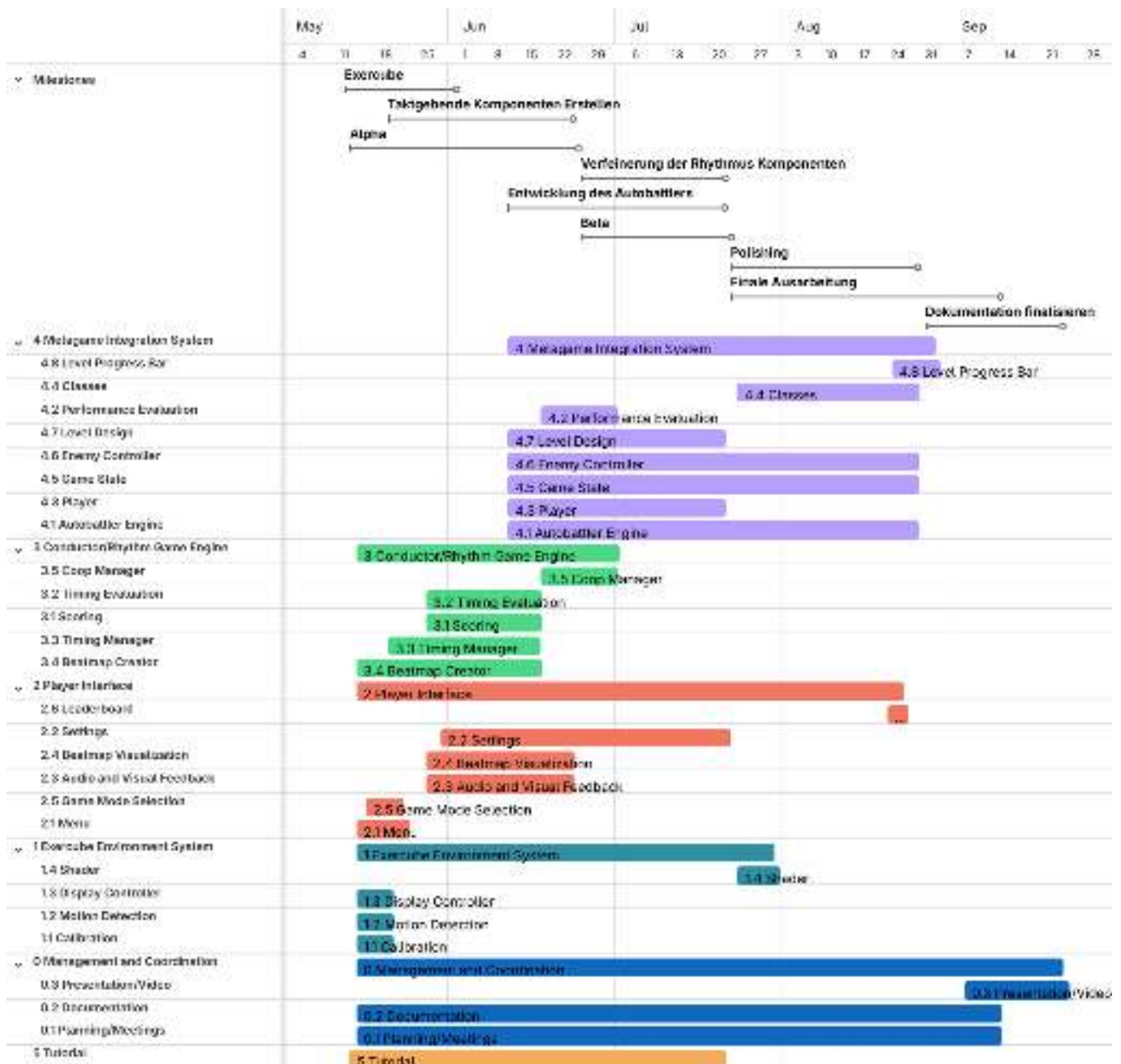


Abbildung 2.3.: Gantt-Diagramm

Daran anschließend folgt der Meilenstein **Taktgebende Komponenten Erstellen**, in dem der Fokus auf der Implementierung eines Systems zur Erstellung von Beatmaps und der ansprechenden Visualisierung dieser liegt. Im Verlauf des Meilensteins wird ein dynamisches Punktesystem entwickelt mit geeigneter Schnittstelle für diverse Komponenten. Die Synchronisation aller beteiligten rhythmischen Elemente stellt dabei eine besondere Herausforderung dar und erfordert sorgfältige technische Abstimmung.

Parallel dazu beginnt die Arbeit am Meilenstein **Entwicklung des Autobattlers**, welcher das Fundament für das motivationale Metaspiel legt. Aufbauend auf der Schnittstelle des zuvor implementierten Punktesystem

wird ein System entworfen, das die Leistungen der Spielenden einzigartig aufbereitet.

Unmittelbar nach Abschluss der Grundlagenarbeit rund um die Rhythmuskomponenten erfolgt die **Verfeinerung der Rhythmus-Komponenten**. In dieser Phase liegt der Fokus auf der Ausarbeitung spielerischer Feinheiten, insbesondere auf der Verbesserung der Interaktion und der Rückmeldung an die Spielenden. Ziel ist es eine frustrationsfreie und schön anzusehende Spielerfahrung zu erschaffen.

Zum Ende des Entwicklungsprozesses wird im Meilenstein **Polishing** besonderes Augenmerk auf die Feinabstimmung aller Systeme gelegt. In diesem Abschnitt werden bestehende Komponenten überarbeitet, aufeinander abgestimmt und hinsichtlich Nutzererlebnis, Performance und Gestaltung optimiert.

Abschließend folgt der Meilenstein **Dokumentation finalisieren**, in dem der Projektbericht überarbeitet und abgeschlossen wird. Zusätzlich werden ein Produktpitch mit einem visuell ansprechendem Trailer erstellt, der das Projekt und seine Ergebnisse anschaulich und nachvollziehbar dokumentiert.

3. Konzeption und prototypische Umsetzung

3.1. Beatmaps und deren Erstellung

3.1.1. Konzept

Das grundlegende Konzept einer Beatmap basiert auf der Idee, Audiodaten mit strukturierten Metadaten zu kombinieren, um daraus ein spielbares Level in einem Rhythmusspiel zu erzeugen. Während das Musikstück den zeitlichen Rahmen und das Tempo vorgibt, definieren die zusätzlichen Daten, wann, wo und wie ein Beat im Spiel erscheint und wie er von den Spieler:innen ausgelöst werden soll. Eine Beatmap stellt damit die zentrale Schnittstelle zwischen der Musik und der interaktiven Spielerfahrung dar. Gleichzeitig eröffnet die Erstellung von Beatmaps den Spieler:innen die Möglichkeit, auf eine Vielzahl an Levels zuzugreifen und diese über den Level-Selection-Screen auszuwählen (siehe Abbildung A.22).

Einer der ersten Prototypen dieses Creators entstand in Form eines Sketches, mit dem grundlegende Interaktionen und Layout-Ideen festgehalten wurden. Diese frühe Skizze diente als Grundlage für die spätere Implementierung des Beatmap-Creators. (siehe Abbildung 3.1).

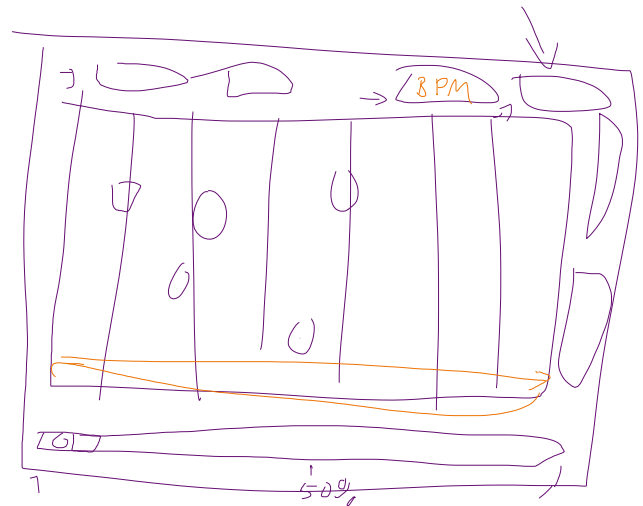


Abbildung 3.1.: Sketch vom Beatmap-Creator
Diese frühe Skizze diente als Grundlage für die spätere Implementierung des Beatmap-Creators. (siehe Abbildung 3.1).

3.1.2. Prototyp

In unserem Ansatz wird die Beatmap in Unity als ScriptableObject realisiert. Um eine Beatmap zu erstellen, wird ein **Beatmap-Creator** eingesetzt, der es ermöglicht, Beats nicht manuell in Rohdaten einzugeben, sondern interaktiv über eine visuelle Oberfläche zu platzieren. Auf diese Weise können die Metadaten direkt entlang der musikalischen Struktur definiert werden, was den Prozess deutlich intuitiver und übersichtlicher macht.

Eine von uns erstellte Beatmap umfasst folgende Daten:

Audio-Clip Der zugehörige Musik- oder Soundtrack, auf dessen Rhythmus die Beatmap basiert. Er wird direkt in Unity als Asset eingebunden und dient als Grundlage für das Timing aller Beats.

BPM Die „Beats per Minute“ geben das Tempo des Songs an. Sie sind essenziell für die korrekte Platzierung der Beats und sorgen dafür, dass Gameplay und Musik synchron ablaufen.

Beats Per Bar Gibt an, wie viele Beats in einem Takt vorkommen. Diese Information wird genutzt, um die Beatmap strukturiert in musikalische Abschnitte zu unterteilen.

Beat-Time Der exakte Zeitpunkt eines Beats innerhalb des Songs, gemessen in Millisekunden. Er bestimmt, wann ein Beat im Spiel ausgelöst wird.

Beat-Height Eine normierte Höhenangabe im Bereich $[0,1]$, die die vertikale Position des Beats im Spielfeld festlegt.

Beat-Spielerzuordnung Definiert, welchem Spieler (Spieler 1, Spieler 2 oder beiden) ein Beat zugeordnet ist. Dies ermöglicht die Gestaltung kooperativer Level.

Beat-Orientation Dieser Wert legt fest, ob der Beat mit der linken oder rechten Hand betätigt werden muss.

Die Daten Audio-Clip, BPM und Beats per Bar werden beim Anlegen der Beatmap im Editor vorab festgelegt und bilden die musikalische Grundlage. Die übrigen Daten entstehen während der Beatmap-Erstellung durch Interaktion mit der Timeline Abbildung 3.2 und dem Popup-Fenster Abbildung 3.3.

Die Timeline visualisiert den zeitlichen Verlauf des Songs und ist in Beats und Sub-Beats unterteilt. Damit ist eine feingranulare Navigation entlang des Musikstücks möglich. Ein Klick auf die Timeline öffnet ein Popup-Fenster und bewirkt gleichzeitig, dass die aktuelle Zeitposition (Beat-Time) sowie die vertikale Klickposition (Beat-Height) automatisch übernommen werden.

Im anschließend erscheinenden Popup werden die restlichen Beat-Attribute abgefragt. Dazu gehören insbesondere die Spielerzuordnung (Player 1, Player 2 oder Both) sowie die Handauswahl (Left oder Right). Über die Buttons Save und Cancel kann die Eingabe entweder bestätigt oder abgebrochen werden. Zusätzlich existiert die Option „Snap-To-Beats“, die dafür sorgt, dass Beats nicht frei auf der Zeitachse gesetzt werden, sondern exakt auf den berechneten Beats oder Sub-Beats des Songs einrasten. Dies erleichtert die Erstellung rhythmisch korrekter Beatmaps erheblich und reduziert das Risiko von ungenauen Platzierungen.

Durch die Kombination aus vorgegebenen Basisdaten, der zeitlichen Navigation über die Timeline und der gezielten Eingabe über das Popup entsteht ein schlanker, aber vollständiger Workflow, um alle notwendigen Beatmap-Daten interaktiv im Editor festzulegen.

Ein zentrales Interface-Element im Beatmap-Creator ist die Scrollbar am unteren Rand des Editors, welche die Länge des Songs repräsentiert. Dieses UI-Element orientiert sich bewusst an etablierten Human-Computer-Interaction-Affordances, da Nutzer:innen bereits mit ähnlichen Bedienelementen aus Anwendungen wie Texteditoren, DAWs (Digital Audio Workstations) oder Media-Playern vertraut sind.

Die Scrollbar bildet die gesamte Songlänge als visuelle Timeline ab. Der bewegliche Schieberegler entspricht dabei einem Fenster auf einen Teilbereich des Songs. Seine relative Breite, die im Editor angepasst werden kann, repräsentiert, wie groß der aktuell sichtbare Abschnitt im Verhältnis zur gesamten Songlänge ist. Dadurch lassen sich gezielte „Zooms“ auf kürzere Songabschnitte durchführen, was ein feingranulareres Erstellen und Platzieren von Beats ermöglicht. Nutzer:innen können so beispielsweise schwierige Passagen eines Songs detailliert bearbeiten, ohne den Überblick über die Gesamtlänge zu verlieren.

Wenn der Song vollständig in das Editor-Fenster passt, wird der Handle automatisch auf die volle Breite gesetzt und die Scroll-Interaktion deaktiviert. Auf diese Weise wird die von Nielsen beschriebene Heuristiken Error Prevention unterstützt.

Darüber hinaus übernimmt die Scrollbar die Rolle eines Navigationswerkzeugs: Durch das Verschieben des Schieberegler wird die Editor-Ansicht entlang der Zeitachse des Songs verschoben. Das ist vergleichbar mit dem horizontalen Scrollen in einem Textdokument oder dem Navigieren auf einer Videotimeline. Damit wird das Beatmap-Editing für die Nutzer:innen vorhersagbar, konsistent und kognitiv entlastend, da auf bekannte Bedienmuster zurückgegriffen wird.

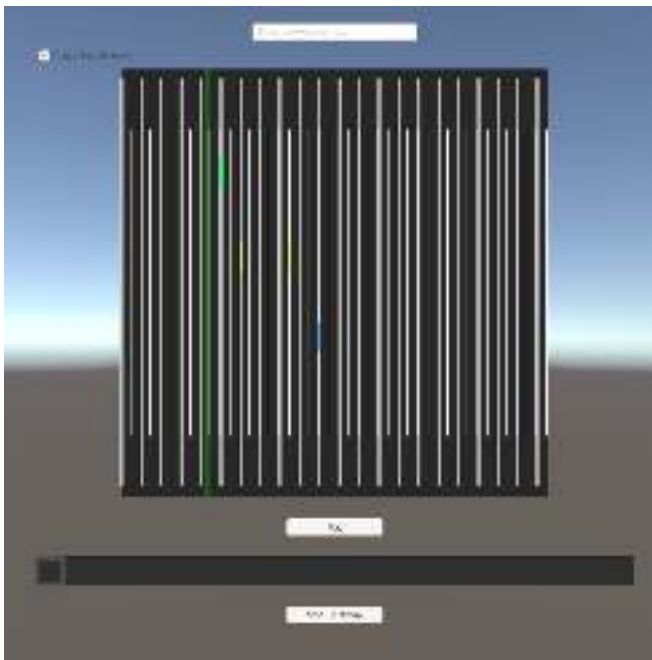


Abbildung 3.2.: Beatmap-Creator mit einigen gespeicherten Beats



Abbildung 3.3.: Beatmap-Creator mit offenem Popup

3.1.3. Methodik

Die initiale Version des Beatmap-Creators wurde als sehr einfacher Prototyp umgesetzt, der ausschließlich Timing-Informationen und die Spielerzuordnung in einer .json-Datei speicherte. Diese frühe Umsetzung hatte das Ziel, schnell eine Testgrundlage zu schaffen, um grundlegende Spielmechaniken zu validieren und ein Gefühl für die notwendige Datenstruktur zu entwickeln. Funktionen wie Handzuordnung, räumliche Positionierung oder die BPM-Synchronisierung waren zu diesem Zeitpunkt noch nicht integriert. Dadurch konnte sich das Team zunächst auf die Kernlogik der zeitlichen Verarbeitung von Beats konzentrieren, ohne von komplexen Editor-Funktionalitäten gebremst zu werden. Im weiteren Entwicklungsverlauf wurde die Beatmap nicht länger als .json-Datei verwaltet, sondern als ScriptableObject in Unity umgesetzt. Zum einen erlaubt diese Methode eine direkte Bearbeitung im Editor, wodurch sich Entwicklungszeiten stark verkürzen und Iterationen am Gameplay schnell umgesetzt werden können. Zum anderen ermöglicht das Arbeiten im Unity-Editor einen einfachen Zugriff auf die im Projekt vorhandenen Assets wie Audio-Clips, wodurch die Synchronisierung von Beat- und Spieldaten unmittelbar überprüft werden kann. Ein dynamisches Laden von Audiodaten zur Laufzeit wurde bewusst vermieden, da dies zusätzliche Herausforderungen wie asynchrones Laden, Caching, mögliche Performance-Einbußen und komplexere Audiokonvertierungen mit sich gebracht hätte. Die Alternative einen externen, eigenständigen Beatmap-Editor mit dynamischem Laden von Audio und Export-/Import-Funktionalität zu entwickeln hätte zwar eine größere Flexibilität für Spieler:innen geboten, jedoch wäre der Entwicklungsaufwand deutlich höher ausgefallen. Zudem wäre die Integration in Unity umständlicher und fehleranfälliger gewesen. Ein weiterer Ansatz wäre die Nutzung externer Tools wie Audacity oder spezialisierte Rhythmus-Game-Editoren, jedoch hätte dies zusätzliche Schnittstellenprogrammierung erfordert, um Datenformate zu konvertieren und in das Spiel zu integrieren. Die gewählte Lösung stellt daher einen Ansatz dar, welcher den Fokus auf schnelle Entwicklung und direkte Integration legt.

3.2. Rhythmusspiel

3.2.1. Konzept

Zielsetzung

Das Rhythmusspiel soll Spieler:innen durch Musik spielerisch zum Sport machen motivieren. Das Ziel ist es, Musik, Bewegung und visuelle Reize zu einem kohärenten Spielerlebnis zu verbinden, das sowohl kognitiv als auch motorisch herausfordert.

Ein besonderer Fokus liegt auf der Gestaltung eines intuitiven und zugleich ästhetisch ansprechenden Interfaces. Zur konzeptionellen Orientierung werden visuelle Elemente und Designprinzipien aus bestehenden Rhythmusspielen analysiert. Die Abbildungen A.1 bis A.3 zeigen die verwendete Sammlung von Interface-Screenshots, die als Inspirationsquelle für die visuelle Struktur, Farbgestaltung sowie Feedbacksysteme des eigenen Spiels verwendet wurden. Bei der Analyse sind zentrale Interface-Muster identifiziert worden, etwa die klare Trennung von Spielfeld und UI-Elementen, die rhythmusbasierte Animation von Objekten sowie die gezielte Verwendung von Kontrasten zur Hervorhebung interaktiver Zonen. Diese Erkenntnisse fließen direkt in die Gestaltung des Spieles, um ein visuelles System zu entwickeln, das sowohl funktional als auch visuell kohärent ist.

Das Konzept sieht auch eine kooperative Dimension vor: Verpasste Beatbuttons können gegenseitig durch Mehraufwand „gerettet“ werden, wodurch Teamarbeit und soziale Interaktion entstehen. Weitere Ideen umfassen Mechaniken wie Spielfeldtausch, bei dem die Spieler:innen koordiniert die Seite wechseln müssen, während sie weiterspielen. Diese kooperativen Elemente sorgen für ein weiteres motivierendes Element des Exergames.



Abbildung 3.4.: Conceptart des Rhythmusspiels, erstellt mit Blender

Spielstruktur

Um die Verletzungsgefahr der Spieler:innen zu minimieren, die sich gemeinsam auf begrenztem Raum schnell bewegen müssen, wird der jeweilige Interaktionsradius dieser auf jeweils eine Seite eingeschränkt. Ihre Bewegungen werden dabei mit VR-Trackern an ihren Händen erfasst. Bei dem Erfassen ist es wichtig, dass nur wirkliche Bewegungen als Interaktion zählt und nicht nur die Position. Kernmechanik ist das präzise Interagieren mit Beatbuttons im Rhythmus der Musik. Diese werden durch die Beatmap vorgegeben und bewegen sich auf eine Beatline zu, die als zeitliche Referenz dient. Ergänzt wird dieses Prinzip durch eine Hitzone, die die Zielposition für die Interaktion markiert. Zur Verstärkung des Feedbacks und Motivation sind zwei zentrale Systeme integriert: Ein **Combo Counter**, der aufeinanderfolgende Treffer zählt und die Punktzahl steigert, sowie eine **Progressbar**, die den Spielfortschritt innerhalb eines Levels visualisiert.

Gestaltungsvarianten (Abbildungen A.4 bis A.7)

Das Konzept zielte darauf ab, zunächst ein funktionierendes Minimum Viable Product in 2D zu erstellen und dieses anschließend schrittweise durch 3D-Visualisierungen zu erweitern. Im Rahmen eines Low Fidelity Prototyping Ansatzes wurden dabei zuerst mehrere Mockups in Blender erstellt. Untersucht wurden dabei die Wirkung unterschiedlicher Formen der Beatbuttons (Würfel, Pyramiden), verschiedener Effekte (z. B. Bloom) sowie ob eine Hervorhebung des nächsten Beatbuttons (Markierung, Ghostbuttons) einen positiven Effekt hat. Die Varianten unterschieden sich im Wesentlichen zwischen stärkerem visuellen Effekt, minimalistischer Reduktion oder zusätzlichem Feedback.

Die Arbeit mit diesen Varianten zeigte, dass ein klar strukturiertes Design mit Farbtrennung und unterstützenden Feedbackelementen (z. B. Hervorhebung des nächsten Beatbuttons, optionale Ghostbuttons) die beste Balance aus Lesbarkeit und Spielerlebnis bietet. Die Variante, welche auf Abbildung A.7 zu sehen ist, wurde als Grundlage für die weitere Entwicklung verwendet.

3.2.2. Prototyp

Der finale Prototyp (Abbildung 3.5) des Rhythmusspiels ist in 3D umgesetzt. Zentrale Elemente des Systems sind die **Hitzones**, die **Beatbuttons** sowie die **Beatline**. Beatbuttons gibt es in zwei Varianten, die jeweils einer Hand zugeordnet sind: Die für die rechte Hand sind blaue Würfel, die für die linke Hand gelbe Prismen. Die Hitzones kennzeichnen die Zonen im Raum, an denen die Spieler:innen mit den Beatbuttons interagieren sollen. Mehrere Sekunden vor der eigentlichen Interaktion wird ein Beatbutton als 3D-Objekt in der Tiefe des Raums erzeugt, das sich entlang einer festen Bahn auf die entsprechende Hitzone zu bewegt. Eine vertikale Beatline scannt den Bildschirm kontinuierlich horizontal und schneidet die Beatbuttons genau in dem Augenblick, in dem sie die Hitzone erreichen (Eine Veranschaulichung ist im Anhang Abbildung A.9 zu finden).

Neben dieser Grundmechanik enthält der Prototyp mehrere Feedback- und Fortschritts-elemente: Ein Punktesystem bewertet jede Interaktion, wobei Spieler:innen 1-100 Punkte erhalten, abhängig wie gut ihre Interaktion getimed war. Ein Combo-System verstärkt diesen Effekt, indem es bei aufeinanderfolgenden Treffern die Punktzahl multipliziert, jedoch wird dies bei Fehlern zurückgesetzt. Eine horizontale Progressbar am oberen Bildschirmrand visualisiert den Fortschritt innerhalb der Beatmap und gibt eine zusätzliche Orientierung über die verbleibende Spieldauer. Darüber hinaus wurde eine **Level Selection** implementiert, die es Spieler:innen ermöglicht, zwischen unterschiedlichen Schwierigkeitsstufen und Musikstücken zu wählen indem ein bestimmtes Level gewählt wird. Dadurch können Spielintensität und Trainingseinheit flexibel an individuelle Bedürfnisse angepasst werden.

Die Interaktionen der Spieler:innen wird in Echtzeit mit HTC Vive-Trackern erfasst. Dabei wird bei schneller Bewegung der Handtracker mittels AABB-Test überprüft, ob diese sich im Bereich eines der Beatbuttons befindet und der entsprechende Button dann ausgelöst.



Abbildung 3.5.: Finaler Prototyp des Rhythmusspiels

3.2.3. Methodik

Die Umsetzung des Spiels erfolgt einem iterativen Entwicklungsansatz, bei dem schrittweise neue Funktionen integriert werden. Beginnend mit dem 2D-Prototyp wurde das System sukzessive um eine 3D-Darstellung, Handzuordnung und der Hitzone erweitert. Diese Vorgehensweise ermöglicht ein schnelles Prototyping, bei dem unterschiedliche Features zunächst in einfacher Form überprüft werden können, um früh festzustellen, ob die zugrunde liegende Idee funktioniert, bevor einzelne Komponenten im Detail ausgearbeitet und in das Gesamtsystem integriert werden. Geplante kooperative Elemente konnten aufgrund von Zeitrestriktionen nicht umgesetzt werden.

Die Beatline wurde pro Display als eigenes Prefab instanziiert, um eine konsistente Sichtbarkeit aus allen Perspektiven sicherzustellen. Für das visuelle Feedback kommt ein Materialsystem mit Emission und Transparenz zum Einsatz, das die Klarheit erhöht. Alternativ wurden zentrale Mesh- oder Shader-Lösungen sowie eine UI-Integration diskutiert, die jedoch jeweils Einschränkungen in Darstellung, Flexibilität oder Performance mit sich bringen.

Ein weiterer zentraler Bestandteil ist der Combo Counter, der aufeinanderfolgende erfolgreiche Treffer der Spieler:innen zählt. Dieses System verstärkt die Rückmeldung durch steigende Multiplikatoren und visuelle Effekte, wodurch sowohl die Motivation als auch der Spielfluss gefördert werden. Fehlerhafte Eingaben setzen den Zähler zurück, was zusätzliche Spannung und Herausforderung schafft und somit den Anspruch steigert, im Rhythmus zu bleiben. Geplant war zudem, erweiterte Feedbackelemente wie auffällige Animationen beim Erreichen neuer Combostufen zu integrieren, vergleichbar mit Effekten, die in anderen Rhythmusspielen als visuelles Highlight kurz „aufpoppen“. Aufgrund von Zeitrestriktionen konnte diese Erweiterung bisher jedoch nicht umgesetzt werden.

Die Progressbar wurde in zwei Varianten (Abbildung A.8) konzipiert, letztlich aber nur in einer minimalistischen, segmentierten Dreiecksdarstellung umgesetzt. Zusätzlich dass sie sich mit Ablauf des Liedes auffüllt, blinkt sie synchronisiert mit der Musik.

Während der Entwicklung wurden mehrere visuelle Mockup-Varianten in Blender erstellt (vgl. Abbildungen A.4 bis A.7). Dabei wurden unterschiedliche Formen, Farben, Effekte und Markierungssysteme getestet. Letztlich stellte sich heraus, dass die Variante mit **Beatline, Beatbuttons in Form von Prismen und Würfeln** sowie einem **Bloom-Effekt**, jedoch **ohne finale gelbe Markierung** und **ohne Ghostblock**, dafür aber mit einer **klar definierten Hitzone**, die beste Umsetzung und Sichtbarkeit erzielte. Besonders wichtig war, essenzielle Informationen nicht ausschließlich durch Farben zu vermitteln, sondern zusätzlich über Form, Position und Effekte, gemäß der Game Accessibility Guidelines von Ellis et al. (n. d.). Diese Gestaltung vereint eine hohe visuelle Prägnanz durch den Leuchteffekt mit einer klaren Orientierungshilfe durch die Hitzone, ohne die Spieler:innen mit zusätzlichen Markierungen oder Overlays zu überlasten.

Anfangs wurde erwogen, die Logik auf der 2D-Variante zu belassen und nur die 3D-Visualisierung anzuzeigen, um die Umsetzung zu vereinfachen und schnelle Iterationen zu ermöglichen. Letztlich erwies sich jedoch die aktuelle Implementierung als praktikabler und besser an die Systemanforderungen angepasst.

3.2.4. Evaluation

Parallel zur Entwicklung des Prototyps wurde eine Evaluation geplant, die zentrale Aspekte des Spielerlebnisses systematisch untersucht. Grundlage bildet der GEQ, ein **standardisierter und vielfach validierter Fragebogen**, der eine zuverlässige und vergleichbare Erfassung des subjektiven Erlebens ermöglicht. Ergänzend dazu wurden Cyberquest-spezifische Fragen entwickelt (Abschnitt A.4). Zusätzlich erfolgt ein detailliertes Logging der Spielerinputs, um mögliche Verzerrungen oder Abweichungen in den Eingabedaten zu erkennen. Für die Evaluation werden die Mockups aus Abbildungen A.4 bis A.7 als Varianten umgesetzt.

Der Ablauf ist als *within-subject*-Design angelegt: Nach Zustimmung zur Datenschutzaufklärung und dem Ausfüllen eines Fragebogens zu anonymisierten Daten (z. B. Alter, Geschlecht, Größe, Gewicht, Erfahrung mit Videospielen, Sport und Musik) spielen die Proband:innen in Paaren dasselbe Level mehrfach in verschiedenen Varianten. Die Reihenfolge wird nach dem Prinzip des *balanced latin square* festgelegt, um Lerneffekte zu minimieren. Während des Spielens erfolgt eine strukturierte Beobachtung anhand vorgegebener Kriterien (Unterabschnitt A.4.1). Direkt nach Abschluss eines Levels füllen die Teilnehmenden den erweiterten GEQ aus. Auf diese Weise werden folgende Aspekte evaluiert:

Immersion durch Bewegung: Über den GEQ werden die Dimensionen „Immersion“ und „Flow“ erfasst. In Kombination mit zusätzlichen Fragen (Unterabschnitt A.4.2) lässt sich prüfen, inwieweit körperliche Bewegung die Immersion verstärkt.

Motivation: Die Bereitschaft, das Spiel erneut zu spielen, wird durch ergänzende Fragen erhoben (Unterabschnitt A.4.3).

Visuelle Verständlichkeit: Beobachtungsdaten (Unterabschnitt A.4.1) und Fragen zur visuellen Wahrnehmung (Unterabschnitt A.4.4) geben Aufschluss über die Verständlichkeit der visuellen Elemente.

Koordination durch Button-Trennung: Über Fragen zur Handtrennung (Unterabschnitt A.4.5) sowie Beobachtungen wird untersucht, wie gut die Steuerungsstruktur gezielte Bewegungskontrolle unterstützt.

Visuelles Feedback und Orientierung: Durch den Vergleich der Varianten wird die Wirkung der visuellen Gestaltung auf die Proband:innen anhand des GEQ-Bereichs „Positive Affect“ evaluiert.

Technische Zuverlässigkeit: Fragen zur Eingabeverfolgung (Unterabschnitt A.4.6) erfassen die Stabilität und Genauigkeit der technischen Umsetzung.

Subjektives Spielerlebnis: Der GEQ liefert standardisierte Werte zu wahrgenommener Herausforderung, Zufriedenheit und Frustration.

Aufgrund Zeitrestriktion konnte die Evaluation nicht durchgeführt werden.

3.3. Metaspiel

3.3.1. Konzept

Laut Costiuc (2019) ist ein Metaspiel „ein Spiel jenseits eines Spiels“. In *Cyberquest* ergänzt es das Kern-Rhythmusspiel, schafft langfristige Motivation und ist so gestaltet, dass Feedback schnell erfassbar und die Steuerung indirekt bleibt.

Um diese Anforderungen zu erfüllen, entschieden wir uns für einen **Autobattler** mit einfachen **Roguelike-Elementen**. Durch die Autobattler-Mechanik wird das Ziel der indirekten Steuerung erfüllt und die Roguelike-Elemente geben eine langfristige Motivation.

Eine große Herausforderung bei der Entwicklung des Metaspiels war es ein intuitives und leicht erfassbares Interface zu gestalten. Dafür wurden mehrere Brainstorming-Sitzungen abgehalten, in denen unterschiedliche Varianten diskutiert wurden. Verworfen wurde beispielsweise ein Ansatz bei dem das Interface angelehnt an bekannte Mobile-Games wie *Raid Shadow Legends* (Abbildung A.11) designet wurde. Dabei hätte jeder Charakter eine eigene Lebensanzeige erhalten und Trefferfeedback wäre in Form von schwebenden Schadenszahlen umgesetzt worden (Abbildung A.12). Ein weiterer verworfener Ansatz war es Schaden durch visuelle Veränderung der Charaktermodelle darzustellen, wie das Fehlen von Gliedmaßen (Abbildung A.13). Beide Varianten wurden als kognitiv zu überladen eingeschätzt. Die Entscheidung fiel auf eine stark reduzierte Lösung, bei der nur zwei Lebensleisten angezeigt werden, eine für die Avatare der Spieler:innen und eine für die Gruppe der Gegner. Auf weitere Elemente, wie schwebende Schadenszahlen wurde bewusst verzichtet. Das Konzept sah vor bei der Entwicklung auf die modulare Erweiterbarkeit zu achten, falls im Laufe der Entwicklung mehr Tiefe gewünscht werden sollte.



Abbildung 3.6.: Conceptart, Dall-E

Das Finale Konzept:

Es wird ein visuell ansprechendes *Minimum Viable Product* entwickelt, welches funktional ist und bei Bedarf einfach erweitert werden kann. Zu Beginn einer Session wählen die Spieler:innen einen von mehreren vorgefertigten Charakteren sowie eine Waffe aus. Anschließend erkunden die beiden Charaktere eine endlose, virtuelle Cyberpunk-Stadt (Abbildung A.10). Dort treffen sie automatisch auf verschiedene Gegnertypen, woraufhin ein automatisierter, rundenbasierter Kampf startet. Die Interaktion der Spieler:innen erfolgt ausschließlich indirekt: Jede gute Leistung im Hauptspiel, wie das Erreichen einer bestimmten Kombo-Kette, belohnt das Metaspiel mit stärkeren Angriffen oder speziellen Fähigkeiten. Charaktere und Waffen dienen dabei rein kosmetischen Zwecken, um den Entwicklungsaufwand überschaubar zu halten. Am Ende jedes durchgespielten Levels, erhalten die gewählten Charaktere Belohnungen, abhängig von der Anzahl und Stärke der besiegten Gegner. Feedback über den Gamestate erhalten die Spieler:innen über die beiden Lebensbalken am oberen und unteren Bildschirmrand.

3.3.2. Prototyp

Umgebung

Die Umgebung stellt eine endlos, prozedural generierte Stadt im Cyberpunk Stil dar. Konkret ist es eine gerade Straße, die von verschiedenen Hochhäusern mit einer variablen Stockwerkanzahl gesäumt wird (Abbildung A.16). Die Hochhäuser sind mit diversen Schildern und anderen Objekten dekoriert, die auf Abbildung A.17 dargestellt sind. In der Luft fliegen die auf Abbildung A.18 sichtbaren fliegenden Autos die Straße entlang. Alle Elemente der Stadt werden in den auf Abbildung A.19 beispielhaft dargestellten, 24 verschiedenen Farben erstellt. Die vorgefertigten Automodelle werden mit passenden Partikeleffekten kombiniert und die Schilder durch einen neu erstellten Shader animiert. Das Ende der „endlosen“ Straße wird durch einen Nebel-effekt versteckt und die Stadt ist in einen Regenschauer eingehüllt, der durch diverse Neon-Lichter durchbrochen wird.



Abbildung 3.7.: Eine Beispielszene des Metaspiels

Die prozedurale Generierung nutzt ein Objekt-Pooling-System: Objekte werden so selten wie möglich neu erstellt und ansonsten wiederverwendet. So kann eine endlose wirkende Stadt dargestellt werden, obwohl nur 60 Objekte einmalig neu erstellt werden. Bei jeder Wiederverwendung werden alle Attribute (Stockwerkszahl, Geschwindigkeit und Frequenz der Autos, Variante und Farbe) neu zufällig bestimmt. Dadurch entstehen etwa 10^{468} Varianten, die 40 Gebäude in der Szene anzuordnen¹. Zusammen mit Dekorationen und Autos ist die Wahrscheinlichkeit, eine identische Szene zu sehen, effektiv 0%.

Charaktere

In dem Metaspiel kommen sowohl Spielercharaktere (die Avatare) als auch Gegnercharaktere vor. Visuell unterscheiden sich diese nicht voneinander. Es gibt 38 verschiedene Charaktermodelle (Abbildung A.20). Jeder Charakter verwendet eine Waffe und hat eine Lauf- und eine Idle- Animation. Die Avatare können verschiedene Waffen tragen, die Gegner nur die Standardausführung. Wie bei der Umgebung ist es auch bei den Avataren möglich verschiedene Farbvarianten auszuwählen. Die Avataranpassung ist über ein simples Menü (Abbildung A.21) jederzeit für beide Spieler:innen möglich.

Automatischer Kampf

Das Metaspiel ist ein Autobattler, dessen Verhalten durch eine einfache Statemachine gesteuert wird. Die Statemachine durchläuft die Zustände Gegnergruppe erstellen, Avatare auf Gegnergruppe zulaufen lassen, Gegnergruppe automatisiert bekämpfen. Eine Gegnergruppe ist eine zufällige Auswahl von fünf Gegnern. Jeder

¹Die Zahl der Varianten berechnet sich wie folgt:

$$\left(2 \text{ (normal/scifi)} \cdot 6 \text{ (Erdgeschossvarianten)} \cdot 24 \text{ (Farben)} \cdot \sum_{k=4}^{19} 3^k \text{ (Stockwerksvarianten)} \right)^{40}$$

Gegner hat die Attribute Leben, Modell und sein Schwierigkeitslevel, welche für die Gruppe kombiniert werden um einen Encounter zu erstellen. Diese fusionierte Entität wird dann in der Kampfsimulation verwendet. Eine Besonderheit des Verhaltens ist, dass beim Zustand Avatare auf Gegnergruppe zulaufen lassen, genau genommen nicht die Avatare bewegt werden, sondern alles außer die Avatare.

Das aktuelle Leben der Avatar- und Gegnergruppe wird mit zwei Lebensbalken dargestellt. Während der Kampfsimulation greifen beide Gruppen nacheinander alle drei Sekunden an. Sowohl die Avatare als auch die Gegnergruppe können Bonusschaden ansammeln: Die Avatare, indem sie Punkte sammeln, und die Gegner, wenn Spieler:innen Beatbuttons verpassen, wobei die Höhe des Bonusschadens vom Level der Gegner abhängt. Sobald die Gegnergruppe keine Lebenspunkte mehr besitzt, wird wieder der erste Zustand der Statemachine ausgelöst, ohne dass die Gegnergruppe zurückschlagen darf.

3.3.3. Methodik

Entwicklungsprozess

Der Prototyp wurde iterativ erstellt, was eine dynamische Anpassung der Arbeitslast ermöglichte. Innerhalb des Prozesses wird ein Konzept mittels fehlender Meilensteinen definiert, einer der Meilensteine prototypisch implementiert, die Implementierung evaluiert und dann ein neues Konzept definiert. Bei der Konzeption wurde insbesondere auch der Fortschritt des gesamten Spieles betrachtet, damit die Arbeitszeit, welche für das Metaspiel aufgewendet wird, dynamisch angepasst werden kann. Beispielsweise wurde das Verhalten der Charaktere zunächst in kleinere Schritte unterteilt, was ein komplexeres Kampfsystem ermöglicht hätte, dies wurde jedoch aufgrund begrenzter Zeitressourcen und verfügbarer Arbeitsstunden vereinfacht. Die umgesetzten Meilensteine sind Umgebung erstellen, Charaktere hinzufügen und das Kampfsystem erstellen.

Besonders auffällig ist das Fehlen der ursprünglich geplanten Roguelike-Elemente in den Meilensteinen, die im finalen Konzept eingeplant waren. Aufgrund mangelndem Fortschritt des Rhythmusspieles wurde dieser Meilenstein gekürzt. Dieses Vorgehen unterstreicht die Wichtigkeit des dynamischen, iterativen Prozesses.

Umsetzung der Mechaniken und mögliche Alternativen

Die Umgebung wurde durch prozedurale Generierung in Kombination mit Objekt-Pooling umgesetzt, um eine performante und endlos erkundbare Stadtumgebung zu schaffen. Die prozedurale Generierung wurde wegen ihrer einfachen Umsetzung der Endlosigkeit verwendet. Einzigartige, handgefertigte Städte pro Lied hätten ein stimmigeres und detaillierteres Stadtbild ergeben, allerdings hätte sich die Stadt dann zwangsweise irgendwann wiederholt. Eine hybride Mischung aus prozeduraler und manueller Generierung durch Parametrisierung der verwendeten Algorithmen ist im aktuellen Prototypen nicht umgesetzt, da zum Zeitpunkt der Implementierung noch nicht klar war, wie genau die Inhalte der Beatmaps strukturiert sein würden. Der Einsatz von Objekt-Pooling statt eines klassischen Erschaffen-und-Löschen-Zyklus wurde gewählt, um eine konstante Performance sicherzustellen. Da das Metaspiel die Leistung des Hauptspiels nicht beeinträchtigen sollte, bot diese Technik einen unkomplizierten und ressourcenschonenden Ansatz ohne nennenswerten Mehraufwand.

Für das automatisierte Kampfsystem wurde eine einfache Statemachine verwendet, da diese Abbildung von künstlicher Intelligenz einfach umzusetzen ist. Komplexere Alternativen wie Verhaltensbäume, einzelne Statemachines für jeden Gegner oder sogar ein Machine-Learning Ansatz wurden von Beginn an aufgrund ihrer explodierenden Komplexität ausgeschlossen, da das Ziel war, ein Minimum Viable Product zu erschaffen.

Bei der Konzeption und Implementation der Statemachine wurde besonders Wert auf Modularität und Erweiterbarkeit gelegt, um das System flexibel für zukünftige Erweiterungen zu gestalten.

Gegnergruppen werden vom Kampfsystem als eine Entität betrachtet, um ausgewogenes und leicht anpassbares Balancing zu gewährleisten. Dieser Ansatz bietet viel Potential für Erweiterungen, wie das dynamische Erstellen von Gruppen anhand ihres Schwierigkeitslevels. Die Gegner sind einfach durch zusätzliche Parameter zu erweitern, wie etwa Resistenzen und Anfälligkeiten oder unterschiedliche Angriffsgeschwindigkeiten. Diese Flexibilität erlaubt einfache Steigerung der spielerischen Tiefe in späteren Iterationen.

4. Zusammenfassung und Ausblick

4.1. Zusammenfassung

Im Rahmen der Lehrveranstaltung wurde ein kooperatives Exergame mit Fokus auf Erweiterbarkeit entwickelt. Der Prototyp wurde iterativ entwickelt, wobei regelmäßig die angestrebten Ziele neu evaluiert wurden, um die gesetzten Ziele bestmöglich zu erfüllen.

Bei der Konzeption wurde ein innovativer Ansatz entwickelt um dem Exergame weitere motivierende Aspekte hinzuzufügen. Zum einen wurde der Bewegungsteil im Rahmen eines Rhythmusspieles umgesetzt, zum anderen wurde ein Metaspiel eingebaut, welches automatisiert anhand der Performanz im Rhythmusspiel automatisiert abläuft. Die geplanten kooperativen Elemente, welche zusätzlich motivierend wären, und die Roguelike-Elemente des Metaspieles, welche eine langfristige Motivation hinzugefügt hätten, mussten aufgrund von zeitlichen Einschränkungen gestrichen werden.

Im Rahmen des Projektes wurde ein Beatmap-Creator entwickelt, ein Werkzeug um Liedern strukturierte Metadaten zuzuordnen, sogenannte Beatmaps, welche dann im Rhythmusspiel verwendet werden können.

4.1.1. Beatmap Creator

Der Beatmap Creator ist ein für das Erstellen der Beatmaps maßgeschneidertes Werkzeug, welches innerhalb des Unity-Editors verwendbar ist. Das Werkzeug erlaubt es strukturierte Metadaten wie die BPM des Liedes und zu welchen exakten Zeitpunkten im Lied auf welcher exakten Höhe ein Beatbutton erscheinen soll zu definieren. Dabei wird für jeden Beatbutton individuell festgelegt, ob er Player 1, Player 2 oder beiden zugeordnet werden soll und mit welcher Hand er ausgelöst werden muss.

Bei der Entwicklung des Werkzeuges wurde insbesondere auf eine effiziente Nutzung von Affordance im Design geachtet. Beispielsweise wurde ein Slider designed, mithilfe dessen man im Lied hin und her springen kann, nach dem Vorbild einer Scrollbar.

4.1.2. Spiel

Es entstanden zwei untrennbar miteinander verflochtene Spiele: Zum einen das Hauptspiel, ein Rhythmusspiel, und zum anderen ein Metaspiel in Gestalt eines Autobattlers. Spieler:innen spielen das Rhythmusspiel aktiv, wodurch sie sich körperlich Betätigen, währenddessen wird ihre Performanz bewertet und anhand dieser Werte der Autobattler indirekt gesteuert.

Rhythmusspiel

Für die Gestaltung des Rhythmusspiels wurden verschiedene Referenzbeispiele untersucht, um Elemente zu identifizieren, die sich optimal für den Exercube eignen. Dabei lag der Fokus insbesondere auf der Passung zwischen körperlicher Betätigung und Spieldesign. Zudem wurde bereits im Entwurfsprozess auf schnell erkennbare Formen und Farben geachtet, um eine hohe Visual Clarity sicherzustellen.

Im Rahmen der Rhythmusspiel-Prototyp Entwicklung wurde eine Darstellung der Beatmap, das Erfassen der Interaktion und die Evaluation der Performanz der Spieler:innen entwickelt. Im Rhythmusspiel wird jeder festgelegte Button der Beatmap als ein für diesen Typen passendes 3D-Objekt dargestellt. Die Objekte werden, damit Spieler:innen korrekt darauf reagieren können, eine gewisse Zeit vor der geplanten Interaktion erstellt und angezeigt. Für jedes 3D-Objekt wird eine Hitzone erstellt, die den Ort der geplanten Interaktion darstellt. Auf diesen Ort bewegen sich die Beatbuttons zu, während eine Beatline das Spielfeld horizontal abscannt. Wenn die Beatline die Hitzone berührt, erreicht der Beatbutton auch die Zone, dies zeigt den korrekten Zeitpunkt der Interaktion an. Abhängig von der Genauigkeit des Timings erhalten die Spieler:innen Punkte, multipliziert mit dem aktuellen Combo-Wert, der durch konsequente Treffer aufgebaut wird. Die Interaktion ist mithilfe zweier HTC-Vive Controllern an den Händen umgesetzt: Durch schnelle Bewegung wird eine Interaktion ausgelöst, falls die Hand sich nun in einem Bereich mit Beatbutton befindet, wird dieser ausgelöst.

Metaspiel

Das Metaspiel ist ein Autobattler, in dem die Spieler:innen automatisiert gegen Gegnergruppen kämpfen. Die Kämpfe finden in einer endlosen, prozedural generierten Cyberpunk Stadt statt. In den Kämpfen fügen sich die beiden Parteien rundenbasiert Schaden, abhängig von der Performanz der Spieler:innen im Rhythmusspiel zu. Für eine hohe Punktzahl und erfolgreich ausgeführte Interaktionen, erhalten die Avatare Bonusschaden, wenn hingegen Beatbuttons nicht getroffen werden, erhalten die Gegner Bonusschaden. Das Interface wurde bewusst minimalistisch gehalten um das Risiko einer kognitiven Überlastung zu minimieren.

4.1.3. Methodik

Um den iterativen Ansatz zu gewährleisten wurden wöchentlich Meetings abgehalten, deren Protokolle unter Abschnitt A.6 einzusehen sind. Dabei wurden im Team der aktuelle Fortschritt besprochen und eventuelle Probleme eruiert, daraufhin wurde der Plan für die nächste Woche aufgestellt. Durch die Meetings wurde ein Informationsfluss zwischen den Teammitgliedern gewährleistet und diverse Probleme konnten, wenn Hilfe eingefordert wurde, sofort behoben werden.

Durch das iterative Vorgehen wurde auch der Projektplan von Abbildung 2.2 durchgehend angepasst wodurch ein aktualisierter Projektplan (Abbildung A.23) entstanden ist. Der Fokus beim Anpassen lag darauf spezialisierte Entwickler für diverse Bereiche zu haben, da so ein effektives Vorgehen gewährleistet werden konnte.

Besonders hervorzuheben ist, dass die meisten Teammitglieder zu Beginn des Projekts nur geringe bis keine Vorerfahrung mit Unity hatten. Im Laufe der Arbeit eigneten wir uns die notwendigen Kenntnisse in Unity sowie den damit verbundenen Entwicklungsprozessen an. Dadurch war das Projekt nicht nur inhaltlich, sondern auch in technischer Hinsicht eine wertvolle Lern- und Praxiserfahrung.

4.2. Lessons Learned

Im Projektverlauf wurde deutlich, dass wir uns in der Planung zu viel vorgenommen hatten und die ambitionierten Ziele innerhalb des gegebenen Zeitrahmens nicht vollständig realisierbar waren. Zwar konnten zentrale Spielelemente erfolgreich umgesetzt werden, doch zusätzliche Features wie die kooperativen Mechaniken mussten aus Zeitgründen zurückgestellt werden.

Ein weiterer zentraler Punkt war die Arbeit mit dem ExerCube. Die technische Integration stellte sich als deutlich komplexer heraus als erwartet. Insbesondere das Debuggen war durch den umständlichen Prozess des manuellen Bauens, Kopieren und Testens direkt am Gerät zeitaufwendig und fehleranfällig.

Diese zusätzlichen Aufwände waren in der ursprünglichen Planung nicht einkalkuliert und führten zu spürbaren Verzögerungen.

Insgesamt haben wir gelernt, den Umfang realistischer einzuschätzen, frühzeitig Prioritäten zu setzen und technische Rahmenbedingungen stärker in die Zeitplanung einzubeziehen. Für zukünftige Projekte wäre es sinnvoll, früh Prototypen direkt auf der Zielplattform zu testen und einen größeren Puffer für unvorhergesehene technische Schwierigkeiten einzuplanen.

4.3. Ausblick

Es gibt eine Vielzahl an möglichen Erweiterungen von Cyberquest: Am Einfachsten ist das Hinzufügen von neuen Liedern und das Erstellen von passenden Beatmaps. Weitere Möglichkeiten bietet das technische und spielerische Weiterentwickeln des Spiels und Beatmap-Creators: Im Creator könnte das manuelle Erstellen der Beatmaps durch frequenzbasierte, automatisierte Vorerstellen exploriert werden, zudem könnte er den Spieler:innen zur Verfügung gestellt werden, sodass diese Player-made Content generieren können.

Im Spiel können die gestrichenen, motivierenden Elemente entwickelt werden, dies umfasst zum einen kooperative Elemente des Rhythmuspiels, zum anderen Roguelike-Elemente des Metaspiels. Das Rhythmuspiel könnte durch Erweiterungen der Beatbuttons verbessert werden, indem es verschiedene Typen gibt, die man beispielsweise durch unterschiedliche Bewegungsabläufe aktivieren muss, oder die im Metaspiel besondere Effekte haben. Naheliegend ist außerdem das Ausführen der Evaluation und das Auswerten und Aufarbeiten von dieser. Dem Metaspiel könnte man eine beliebige Tiefe neben den Roguelike-Elementen geben, wie beispielsweise ein Klassen- und Waffen-System, sodass diese Auswahl nicht nur kosmetischer Natur ist. Es könnten auch andere Stadt-Themen erstellt werden, wie Mittelalter, und die prozedurale Generierung könnte mithilfe der Beatmaps parametrisiert werden um lieder spezifische Stadt-Unterarten zu erstellen.



Literatur

- Costiuc, S. (2019, 12. Februar). *What Is A Meta-Game?* Informa TechTarget aka Game Developer. Verfügbar 7. März 2025 unter <https://www.gamedeveloper.com/design/what-is-a-meta-game->
- Ellis, B., Ford-Williams, G., Graham, L., Grammenos, D., Hamilton, I., Games, H., Lee, E., Manion, J., & Westin, T. (n. d.). *Game accessibility guidelines*. Verfügbar 9. Juni 2025 unter <https://gameaccessibilityguidelines.com/ensure-no-essential-information-is-conveyed-by-a-fixed-colour-alone/>

A. Anhang

A.1. Bedienungsanleitung

A.1.1. Vorbereitung

HTC Vive Controller befestigen: Schalten Sie die Controller and und befestigen Sie jeweils einen Controller an jeder Hand mit den vorgesehenen Schlaufen. Achte darauf, dass sie fest, aber bequem sitzen.

Spiel starten: Schalten Sie das ExerCube System ein und starten Sie das Spiel.

A.1.2. Hauptmenü

Level Selection: Hier werden alle verfügbaren Level des Spiels aufgelistet. Der Spieler kann mit den Pfeiltasten oder der Maus das gewünschte Level auswählen und starten.

Character Selection: In diesem Menü können beide Spieler unabhängig voneinander ihre Spielfigur konfigurieren. Zur Auswahl stehen:

- **Character:** Auswahl des Charakters.
- **(Character-)Color:** Anpassung der Charakterfarbe.
- **Weapon:** Auswahl der Waffe.
- **(Weapon-)Color:** Anpassung der Waffenfarbe.

Änderungen werden mit den linken und rechten Pfeiltasten vorgenommen. Mit **Close** wird die Auswahl gespeichert und das Menü geschlossen.

Options: Hier können Anzeige- und Audioeinstellungen angepasst werden:

- **Display:**
 - Mit F1 öffnet sich das Display Setup.
 - Mit F2 kann die Display-Kalibrierung aufgerufen werden.
- **Audio:** Die Lautstärke lässt sich über Schieberegler individuell einstellen:
 - Master Volume:** Gesamte Lautstärke des Spiels.
 - Music Volume:** Lautstärke der Hintergrundmusik.
 - SFX Volume:** Lautstärke der Soundeffekte.

Mit Close werden die Einstellungen gespeichert und das Menü geschlossen.

A.1.3. Das Spiel

Während des Spiels hören Sie Musik. Treffen Sie die Beats im richtigen Rhythmus. Sobald ein Beat-Symbol erscheint, bewegen Sie die entsprechende Hand in die angezeigte Form und „drücken“ Sie im richtigen Timing.

A.1.4. Beatmap-Creator

Öffnen der Szene:

- Navigiere im Projektordner zu `/Assets/Creator/`.
- Öffne die Szene `RhythmGameEditorScene.unity`.

Einstellung der Auflösung:

- Stelle die Resolution auf **1080 x 1080** ein.

Bearbeiten der Timeline-Einstellungen:

- Wähle in der Szene das Objekt `TimelinePanel` aus.
- Öffne im `Timeline Handler Script` die konfigurierbaren Felder.
- Es sollten ausschließlich nur folgende Felder geändert werden:

BPM: Bestimmt die Beats per Minute des Songs.

Beats Per Bar: Gibt an, wie viele Schläge pro Takt angezeigt werden.

Offset: Verschiebt den Song leicht nach vorne oder hinten, um den Takt korrekt zu synchronisieren.

View Range Ms: Legt fest, welcher Abschnitt des Songs in der Timeline angezeigt wird. Ein größerer Wert zeigt einen längeren Ausschnitt mit mehr Beats, ein kleinerer Wert verhält sich ähnlich wie ein Zoom, wodurch ein kürzerer Abschnitt feiner und detaillierter dargestellt wird.

Audio Clip: Hier wird die Audiodatei des Songs eingefügt.

Erstellen einer Beatmap:

- Klicke in der Mitte auf den Canvas, um an einer bestimmten Zeit und Höhe einen Beat zu setzen.
- Es öffnet sich ein Popup, in dem ausgewählt werden kann:
 - Hand:** Links oder Rechts (Zuordnung zu den Händen).
 - Spieler:** Player 1, Player 2 oder Both Player.
- Bestätige die Eingabe mit **Save**, um den Beat in die Beatmap einzutragen.
- Mit **Cancel** kann der Vorgang abgebrochen werden.

Verwaltung der Beatmap:

- Oben im Editor-Fenster kann der Beatmap ein **Name** gegeben werden.

- Mit dem **Slider** am unteren Rand kann innerhalb des Songs gescrollt werden, um verschiedene Abschnitte zu bearbeiten.

Speichern der Beatmap:

- Mit dem **Save**-Button wird die Beatmap als `.asset`-Datei gespeichert.
- Der Speicherort ist `Assets/Resources/beatmaps/`.

A.2. Technisches Manual

A.2.1. Beatmap-Creator

Die Erstellung und Verwaltung von Beatmaps erfolgt über mehrere zentrale Skripte: `BeatDataScriptableObject.cs`, `PopupHandler.cs`, `SliderHandler.cs` und `TimelineHandler.cs`. Sie steuern das Datenmodell, die Benutzerinteraktion, die Timeline-Darstellung sowie den Export.

BeatDataScriptableObject

Zweck `ScriptableObject` zur persistenten Speicherung aller Beatmap-Daten inklusive Metadaten.

Felder

Beats Liste von `BeatData`-Objekten mit Zeit, Position, Typ und Eingabe.

AudioClip Referenz auf die Audiodatei des Songs.

BPM Geschwindigkeit des Songs in Beats per Minute.

BeatsPerBar Anzahl der Schläge pro Takt.

Offset Synchronisationswert, um den Takt mit dem Song abzugleichen.

PopupHandler

Zweck Verwaltung des Eingabedialogs beim Hinzufügen neuer Beats.

Einstiegspunkte

OpenPopup() Öffnet den Dialog für einen temporär gesetzten Beat.

OnSaveClicked() Bestätigt die Eingaben und speichert den Beat.

OnCancelClicked() Bricht ab und entfernt den temporären Marker.

GetBeatmapName() Liefert den vom Benutzer definierten Beatmap-Namen.

SliderHandler

Zweck Steuerung des Timeline-Sliders zur Navigation und zum Zoomen im Song.

Einstiegspunkte

OnSliderChanged() Passt die sichtbare Startzeit der Timeline an.

SyncSliderWidthToTimeline() Synchronisiert die Sliderbreite mit der Länge der Timeline.

UpdateHandleSize() Verändert die Handle-Größe proportional zur Ansicht (vergleichbar mit einem Zoom-Effekt).

TimelineHandler

Zweck Zentrale Steuerung der Timeline: Rendering der Beats, Beat-Erstellung, Audio-Steuerung und Export.

Einstiegspunkte

GenerateAllBeatTimes() Berechnet Beat-Zeitpunkte anhand von BPM.

DrawBeats() Rendert Beatmarker und Sub-Beatmarker im sichtbaren Bereich.

OnPointerClick() Fügt Beats durch Benutzerinteraktion hinzu (inkl. Snap-to-Beat).

PlaySong() / PauseSong() / TogglePlayPause() Steuern das Abspielen des Songs.

ExportToScriptableObject() Exportiert die Beatmap als `BeatDataScriptableObject` in Assets/Resources.

Datenstrukturen

BeatData Struktur für einzelne Beats mit `timeMs`, Höhe, Typ und Eingabe.

BeatCollection Container für eine Liste von Beats und deren Metadaten.

A.2.2. Rhythmusspiel

Das Rhythmuspiel wird durch eine Reihe zentraler Klassen realisiert, die gemeinsam das Verhalten, die Visualisierung und die Interaktion im Spiel steuern. Im Folgenden werden diese Klassen beschrieben.

BeatbuttonController.cs

Zweck Steuert das Verhalten eines einzelnen Beatbuttons. Verarbeitet Spielerinteraktionen, validiert das Timing, gibt visuelles Feedback und löst automatisch eine Miss aus, falls der Button nicht rechtzeitig gedrückt wird.

Struktur

AllowedError maximal zulässige Zeitabweichung.

Data Metadaten zu diesem Beat (Zeit, Seite, Typ).

_Renderer Darstellung des Buttons, inkl. Farbfeedback.

_PointController Referenz zum Punktsystem.

Einstiegspunkte

HandleInteraction() Überprüft Eingaben und bewertet sie als Treffer oder Fehler.

CorrectPress() Verarbeitet einen gültigen Treffer.

FalsePress() Verarbeitet eine Fehleingabe oder das Ablaufen des Buttons.

AutoExpire() Coroutine, die automatisch nach Ablauf der erlaubten Zeit eine Miss registriert.

BeatmapDataProvider.cs

Zweck Stellt Beatmap-Daten für Songs bereit. Enthält Dummy-Daten für Tests und erzeugt Beatbutton-Informationen.

Struktur

BeatmapDataProvider Lädt und liefert **BeatmapData** zu einem Song.

BeatmapData Enthält Metadaten (BPM, BeatsPerBar, Offset) sowie eine Liste an **BeatButtonData**.

BeatButtonData Repräsentiert einen einzelnen Beatbutton mit Zeitstempel, Höhe, Typ und Seite (links/rechts).

Einstiegspunkte

GetBeatmapData() Liefert Beatmap-Daten zu einem Songnamen.

SetDummyData() Erzeugt Testdaten mit variierenden Beatbuttons.

BeatmapVisualizer.cs

Zweck Zentrale Visualisierung der Beatmap. Spawnt Beatbuttons, Geisterblöcke und Markierungen, die synchron zur Musik auf die Beatline zufliegen.

Struktur

BeatButtonPrefab, LeftBeatButtonPrefab Prefabs für die Beatbuttons.

HitZonePlanePrefab Markierungen der Trefferzonen.

BeatbuttonContainer Container für alle visuellen Elemente.

_AvailableBeatButtonObjects Object Pooling zur Performanceoptimierung.

Einstiegspunkte

Update() Haupt-Loop: bewegt die Beatline, spawnt neue Elemente, animiert Bewegungen.

SpawnBeatButtonAndGhost() Erzeugt Button, Marker und ggf. Ghost-Block.

AnimateFlyIns() Animiert Buttons vom Spawnpunkt zur Beatline.

handleButtonPress() Callback bei Button-Interaktion, räumt Elemente auf.

Conductor.cs

Zweck Steuert die Musik und ist Taktgeber für das gesamte Spiel. Berechnet Positionen im Song und Beat, um Spielobjekte zu synchronisieren.

Struktur

CurrentBeatmapData aktuell geladene Beatmap.

SongPositionInSeconds aktuelle Songposition in Sekunden.

BeatsPerBar, BPM, BeatsPerSecond musikalische Metadaten.

Player, AudioSource für Songwiedergabe.

Einstiegspunkte

Update() Aktualisiert kontinuierlich Songposition, BarPosition und Beat-Infos.

CalculateScreenPositionOfBeat() Berechnet Bildschirmpositionen für Beats.

Play() Startet die Wiedergabe eines Songs und lädt Beatmap-Daten.

PointController.cs

Zweck Verwaltet Punkte, Combo-Multiplikator und Interaktion mit dem Kampfsystem.

Struktur

_CurrentPoints Gesamtpunktzahl.

_CurrentCombo aktueller Combo-Multiplikator.

BeatsPerBar, BPM, BeatsPerSecond musikalische Metadaten.

_HitCounter Anzahl aufeinanderfolgender Treffer.

Einstiegspunkte

Hit() Erhöht Combo und Zähler bei einem Treffer.

Miss() Setzt Combo zurück und meldet Fehler an den **FightController**.

AddPoints() Berechnet und addiert Punkte mit aktuellem Multiplikator.

ProgressBar.cs

Zweck Visualisiert den Fortschritt durch den Song in Form einer Reihe von Dreiecken, die sich im Takt füllen.

Struktur

TrianglePrefab Vorlage für die Fortschrittsmarker.

EmptyTriangle, FilledTriangle, FillingTriangle Materialien für Zustände.

_Triangles Liste der erzeugten Dreiecke.

_Conductor Referenz zum Taktgeber.

Einstiegspunkte

Start() Initialisiert und erzeugt die Dreiecke im Layout.

Update() Aktualisiert Fortschritt, füllt Marker und sorgt für Blinkeffekte.

A.2.3. Metaspiel

Umgebung

Die Umgebung wird mittels `EnvironmentController.cs` und `AirtrafficSpawner.cs` gesteuert. In den Settings-SerializeField's können Parameter der Stadt angepasst werden, die Scene-References werden zur korrekten Positionierung verwendet und die Prefabs erlauben das Aussehen der Stadt anzupassen. Es gibt dabei folgende Prefabs

BuildingPrefab Ein leeres Gameobject, um die anderen Prefabs organisiert abzulegen

BuildingPropsPrefab Die Props (Deko) die ab dem 1. Stockwerk hinzugefügt werden können

GroundFloorPrefab Die verschiedenen Erdgeschoss-Gebäude, aufgeteilt in scifi und normal

GroundFloorPropsPrefab Die Props (Deko) für das Erdgeschoss

FloorPrefab Die verschiedenen Gebäude ab dem 1. Stockwerk, aufgeteilt in scifi und normal

RoadPrefab Die Straße

EnvironmentController

Zweck Die Umgebung des Spiels wird durch die Klasse `EnvironmentController` gesteuert, die für die Erstellung, Bewegung und Wiederverwendung von Gebäuden und Straßen zuständig ist. Das System arbeitet mit einem Pooling-Ansatz, bei dem nicht mehr sichtbare Segmente an den Anfang des Spielfelds zurückversetzt und wiederverwendet werden, um eine scheinbar endlose Umgebung zu schaffen.

Einstiegspunkte

Start() Initialisiert die Umgebung. Es findet die Kamera, berechnet die Frustum-Ebenen und instanziiert die anfänglichen Segmente für die Straßen und Gebäude.

Update() Wird in jedem Frame aufgerufen, um die Bewegung der Stadt zu steuern (`MoveCity()`) und die Sichtbarkeit der Objekte zu überprüfen (`CheckObjectVisibility()`).

MoveCity() Bewegt die gesamte Umgebung basierend auf der `CameraSpeed` in Richtung des `FutureZPosition`.

CheckObjectVisibility() Überprüft, ob das erste Segment in den Listen der linken Wand, der rechten Wand oder der Straße außerhalb des Sichtfelds der Kamera liegt.

Reposition...Segment() Eine Reihe von Funktionen, die ein nicht mehr sichtbares Segment an das Ende der Liste verschieben und seine Position neu berechnen.

AddProps() Verwaltet die Wiederverwendung und Platzierung von Requisiten an Gebäuden basierend auf der `chanceForProps`.

InitChildList() Eine Hilfsfunktion, die die anfängliche Liste von Segmenten instanziiert.

GotoNextBattleField() Eine öffentliche Methode, um die `FutureZPosition` zu ändern und die Kamera zu einem neuen Schauplatz zu bewegen.

Segment Die abstrakte Basisklasse für alle wiederverwendbaren Umgebungssegmente (`Building`, `Road`).

CheckVisibility() Überprüft, ob das Segment innerhalb des Kamera-Frustums liegt.

Building Erbt von Segment. Stellt ein Gebäude dar, das aus einem Erdgeschoss und mehreren Stockwerken besteht.

RandomizeBuildingFloors() Weist dem Gebäude zufällig eine Höhe und Textur zu.

Road Erbt von Segment. Stellt ein Straßensegment dar.

VariantCollection Eine Hilfsklasse, die eine Sammlung von visuellen Varianten für ein Segment verwaltet, wie z.B. verschiedene Texturen oder Modelle.

AirtrafficSpawner

Zweck Die Klasse `AirtrafficSpawner` ist für die dynamische Erzeugung, Bewegung und Wiederverwendung von Flugautos in der Unity-Szene zuständig. Sie nutzt ein Object Pooling-System, um Objekte effizient zu verwalten und die Leistung zu optimieren.

Einstiegspunkte

Start() Initialisiert die Object-Pools für fliegende Objekte.

Update() Steuert die Hauptlogik für Bewegung und Spawning.

MoveObjects() Bewegt alle aktiven Flugobjekte vorwärts.

CleanupObjects() Überprüft, ob Objekte die Entfernungszonen (`BehindRemoveBox`, `InFrontRemoveBox`) verlassen haben und gibt sie an den Pool zurück.

TrySpawning() Versucht, neue Objekte in den definierten Spawn-Zonen zu erstellen.

SpawnAirtrafficObject() Wählt ein Objekt aus dem Pool oder erstellt ein neues, platziert es in der Szene und aktiviert es.

Kampfsystem

Die Klassen `Encounter`, `EncounterController`, `EnemyManager`, `FightController`, `CharacterController`, `EntityData` und `EnemyData` steuern das Kampfsystem des Spiels.

Encounter Diese Klasse repräsentiert eine einzelne Begegnung (Kampf) mit einer Gruppe von Gegnern. Sie verwaltet die Gesamtgesundheit, den Schwierigkeitsgrad und den Angriffswert der Gruppe.

Zweck Bündelt alle Informationen und die Logik für eine einzelne Kampfbegegnung.

Einstiegspunkte

TakeDamage(float damage) Verringert die Gesamtgesundheit der Gruppe und löst das Sterben von Gegnern aus.

HandleEnemiesDying() Entfernt den vordersten Gegner, sobald die Gesamtgesundheit unter seinen maximalen Wert fällt.

EncounterController Dieses Skript steuert den gesamten Ablauf einer Begegnung, von der Erzeugung über die Bewegung bis zum Kampf.

Zweck Verwaltet den Status der Begegnung und wechselt zwischen den Phasen `SpawnEncounter`, `MoveToEncounter` und `FightEncounter`.

Einstiegspunkte

Update() Der Haupt-switch-Block, in dem das Verhalten für jeden `EncounterStatus` definiert ist. Hier können neue Phasen oder Aktionen hinzugefügt werden.

EnemyManager Dieses Skript ist für die Erstellung von Gegner-Gruppen für eine Begegnung verantwortlich.

Zweck Lädt alle verfügbaren Gegnerdaten und erzeugt daraus eine zufällige Gruppe von `EnemyInstance`-Objekten an vordefinierten Spawn-Punkten.

Einstiegspunkte

CreateEncounter() Hier wird die Logik zur Auswahl und Instanziierung der Gegner implementiert. Sie können die Zufälligkeit oder die Auswahl der Gegner an dieser Stelle anpassen.

FightController Dieses Singleton-Skript verwaltet die Kernlogik des Kampfes, einschließlich der Schadensberechnung und der Aktualisierung der Lebensbalken.

Zweck Steuert den Kampf zwischen dem Spieler und dem aktuellen Gegner-Encounter.

Einstiegspunkte

Hit(int nextCombo) Erhöht den Spielerschaden basierend auf dem Combo-Wert.

Miss() Erhöht den Schaden, den der Spieler erleidet, basierend auf dem Schwierigkeitsgrad des Gegners.

Attack() Führt die Schadensberechnung für beide Seiten durch. Hier kann die Logik für Schaden und Heilung angepasst werden.

CharacterController Dieses Skript kümmert sich um die visuelle Darstellung des Spielercharakters, wie z.B. das Wechseln von Modellen und Materialien.

Zweck Verwaltet das Aussehen des Charakters und seiner Waffe sowie die Laufanimation.

Einstiegspunkte

Next...() / Previous...() Methoden Steuern das Umschalten zwischen verschiedenen Charakter-, Waffen- und Materialoptionen.

EntityData und EnemyData Dies sind `ScriptableObject`s, die zur Datenspeicherung verwendet werden.

EntityData Eine abstrakte Basisklasse für alle Entitäten im Spiel.

EnemyData Erbt von `EntityData` und enthält spezifische Daten für einen Gegnertyp. Hier werden die Basiswerte für Gesundheit, Schaden und Angriffsgeschwindigkeit von Gegnern definiert.

A.2.4. Tracking

Das Tracking-Modul wird durch die Klasse `PunchTracker.cs` realisiert. Sie verarbeitet die Hand-Tracking-Daten des Spielers, erkennt Punches anhand der Geschwindigkeit und prüft, ob ein Beatbutton getroffen wurde. Zusätzlich steuert sie die visuelle Rückmeldung und die Interaktion mit den Beatbuttons.

PunchTracker.cs

Zweck Verarbeitung der Handpositionen und Erkennung von Punches im Rhythmusspiel.

Struktur

leftHandTracker, rightHandTracker Referenzen auf die Hand-Tracker-Objekte (Transform).

punchSpeedThreshold Mindestgeschwindigkeit, ab der ein Punch erkannt wird.

localPositionCorrection Offset zur Korrektur der Handpositionen.

lastLeftPos, lastRightPos Zwischenspeicher für die Positionen der Hände im letzten Frame.

poseTracker Referenz auf das Tracking-System.

leftIndicator, rightIndicator Visuelle Rückmeldung bei Punches.

Einstiegspunkte

Start() Initialisiert die Tracker-Referenzen und das Tracking-System. Setzt die Startpositionen der Hände.

Update() Holt die aktuellen Handpositionen, berechnet die Geschwindigkeit und prüft, ob ein Punch erkannt wurde. Gibt Debug-Informationen aus.

CheckHandHit(Vector3 handPos, string handLabel) Prüft, ob die Hand einen Beatbutton trifft und löst die Interaktion aus.

A.3. Verwendete Technologien

- Die vorliegende Arbeit wurde eigenständig erstellt und wurde unter Zuhilfenahme von Grammarly/ChatGPT/Gemini/DeepL sprachlich überarbeitet.
- Das Technische Manual entstand, indem der geschriebene Code einer KI übergeben und nach einer Zusammenfassung sowie Einstiegspunkten gefragt wurde. Das Ergebnis wurde gründlich auf Korrektheit geprüft und bei Bedarf manuell angepasst.
- Discord wurde verwendet für interne Kommunikation
- Google Docs, Sharelatex und Miro wurde verwendet um Dokumente kollaborativ zu bearbeiten

A.4. Erweiterung des Game Experience Questionnaire (GEQ)

A.4.1. Beobachtungspunkte

Während der Spielsitzungen werden folgende Aspekte beobachtet und anhand eines standardisierten Rasters dokumentiert:

- **Immersion und Fokus:** Wirkt die Person vertieft und konzentriert oder zeigt sie Ablenkung/Desinteresse?
- **Körperliche Bewegung:** Ausmaß und Intensität der Bewegungen (z. B. große vs. kleine Bewegungen, aktive vs. passive Haltung).
- **Koordination:** Auffälligkeiten bei der Hand- oder Körperkoordination (z. B. Verwechslung der Hände, Timing-Probleme).
- **Soziale Interaktion:** Zusammenarbeit und Kommunikation zwischen den beiden Spieler:innen (z. B. gegenseitige Unterstützung, Rettungsmechaniken).
- **Emotionale Reaktionen:** Sichtbare Freude, Frustration oder Langeweile während des Spiels.
- **Orientierung und Verständnis:** Suchen die Spieler:innen häufiger nach visuellen Hinweisen oder wirken sie unsicher in Bezug auf Position und Timing?
- **Technische Probleme:** Auffälligkeiten wie Erkennungsfehler der Tracker, Eingabelatenzen oder unklare visuelle Rückmeldungen.

A.4.2. Körperliche Aktivität und Spielerlebnis

- Ich hatte das Gefühl, dass meine Bewegungen einen wichtigen Einfluss auf das Spielerlebnis hatten.
- Ich hatte das Gefühl, dass die Bewegung das Gefühl verstärkt haben, in der Musik zu sein?"
- Ich empfand die Bewegungen als angenehm und passend zum Spiel.
- Die Intensität der Bewegungen war für mich herausfordernd, aber machbar.

A.4.3. Motivation

- Ich habe Lust, das Spiel erneut zu spielen.
- Das Spiel hat mich motiviert, mich körperlich zu bewegen.
- Ich würde das Spiel auch außerhalb der Studie spielen.
- Ich fühle mich durch das Punkte- und Combo-System dazu angeregt, mich zu verbessern?
- Der Erfolg im Metaspiel hat mich bei der körperlichen Bewegung motiviert.

A.4.4. Visuelle Verständlichkeit

- Ich wusste jederzeit, wann und wo die nächste Interaktion stattfindet.
- Die visuellen Hinweise waren klar und verständlich.
- Ich konnte die Position der Beatbuttons jederzeit erkennen.
- Das visuelle Feedback (z. B. Leuchten, Hervorhebungen) war hilfreich.

A.4.5. Handtrennung

- Die Zuordnung der Beatbuttons zu den Händen war intuitiv.
- Ich habe mich bei der Handzuordnung häufig vertan.
- Ich habe die Zuordnung der Farben zu den Händen als hilfreich empfunden.
- Ich habe die Zuordnung der Formen zu den Händen als hilfreich empfunden.

A.4.6. Tracking

- Die Eingaben meiner Handbewegungen wurden präzise erkannt.
- Ich habe das Gefühl, dass das Tracking zuverlässig funktioniert hat.

A.5. Weitere Abbildungen

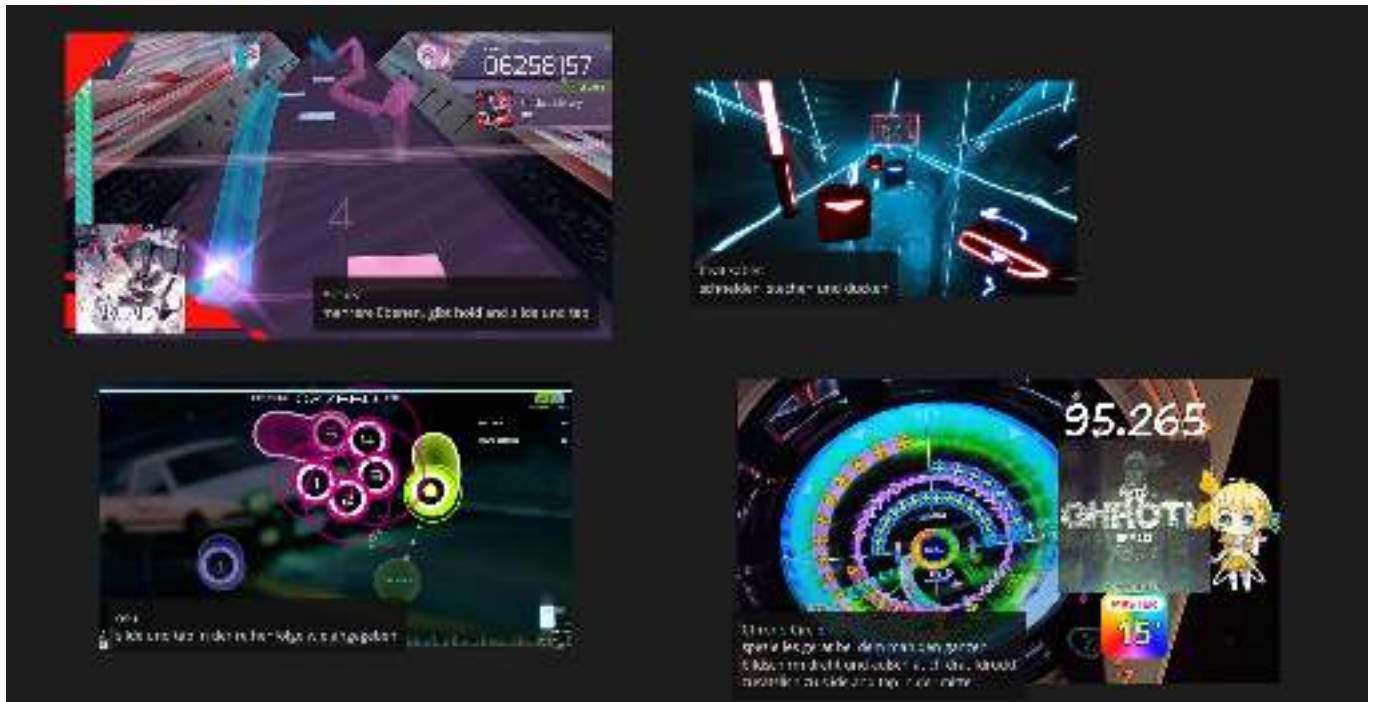


Abbildung A.1.: Inspirationssammlung für das Rhythmuspiel-Interface - Teil 1
 Spiele: *Arcaea*, *Beat Saber*, *Osu* und *Chrono Circle*

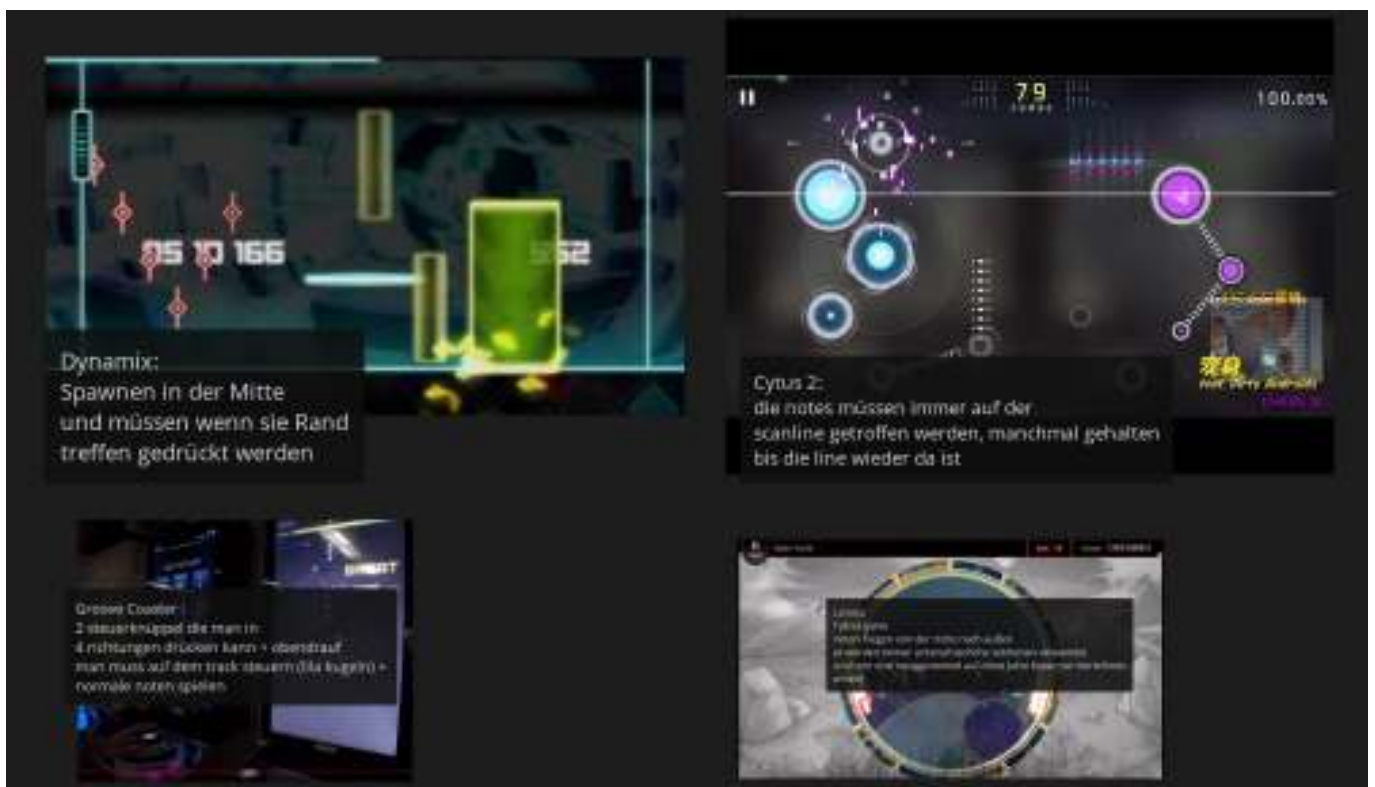


Abbildung A.2.: Inspirationssammlung für das Rhythmuspiel-Interface - Teil 2
 Spiele: *Dynamix*, *Cytus 2*, *Groove Coaster* und *Lanota*

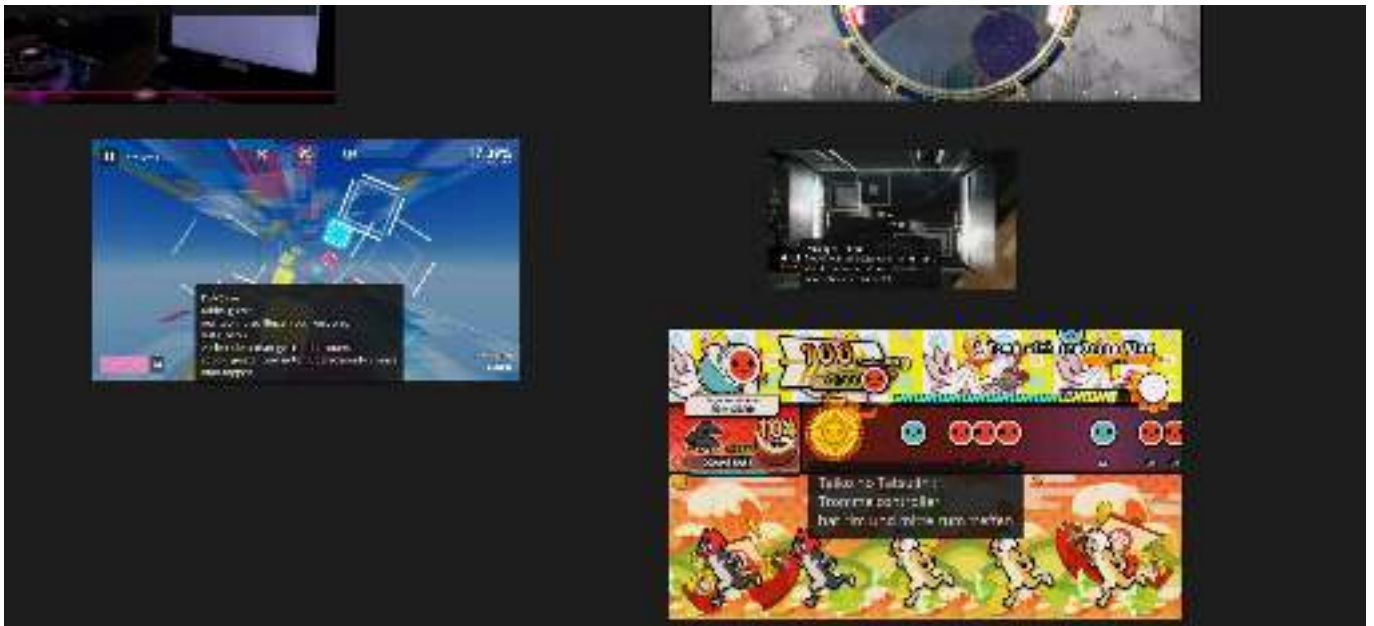


Abbildung A.3.: Inspirationsammlung für das Rhythmusspiel-Interface - Teil 3
 Spiele: *Polytone*, *Taiko no Tatsujin* und *Paradigm Reboot*

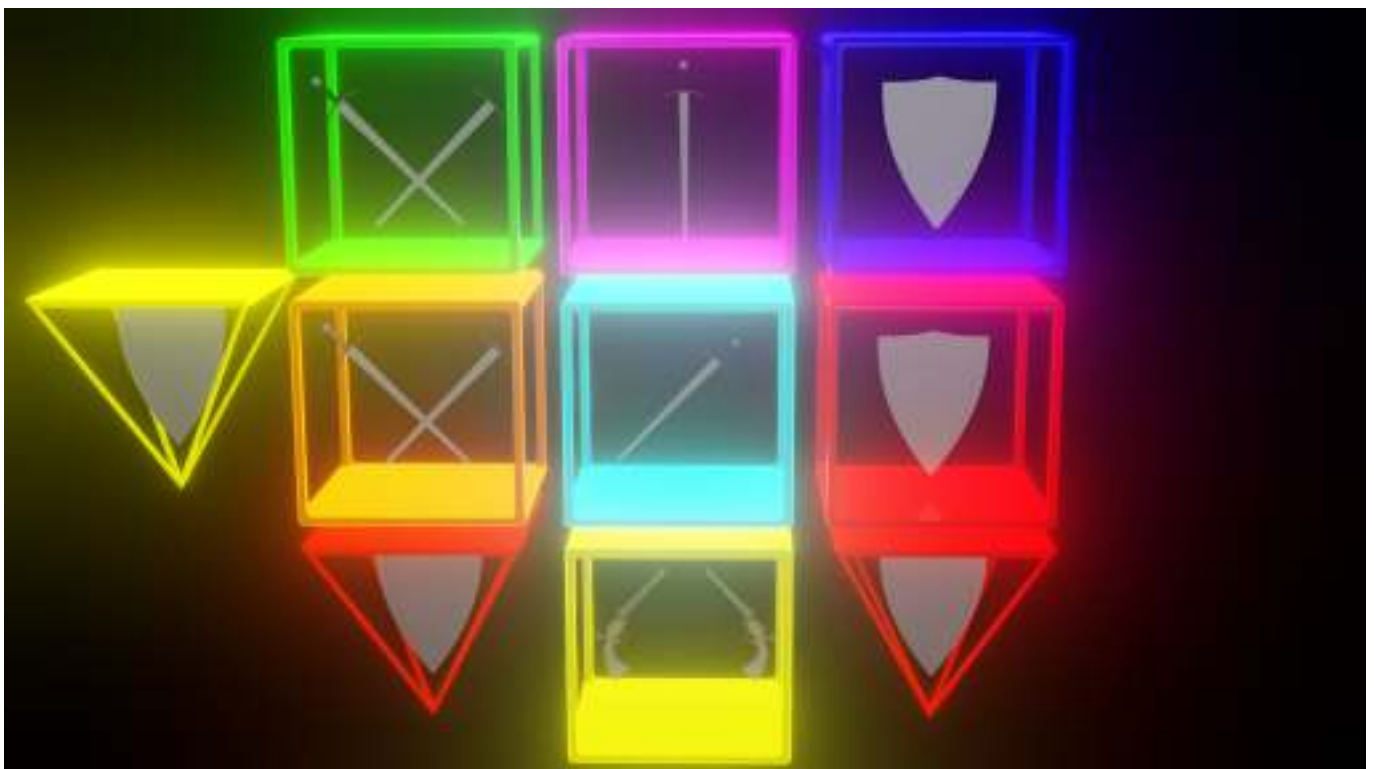


Abbildung A.4.: Ein 3D Mockup der Beatbuttons des Rhythmusspiels - Teil 1

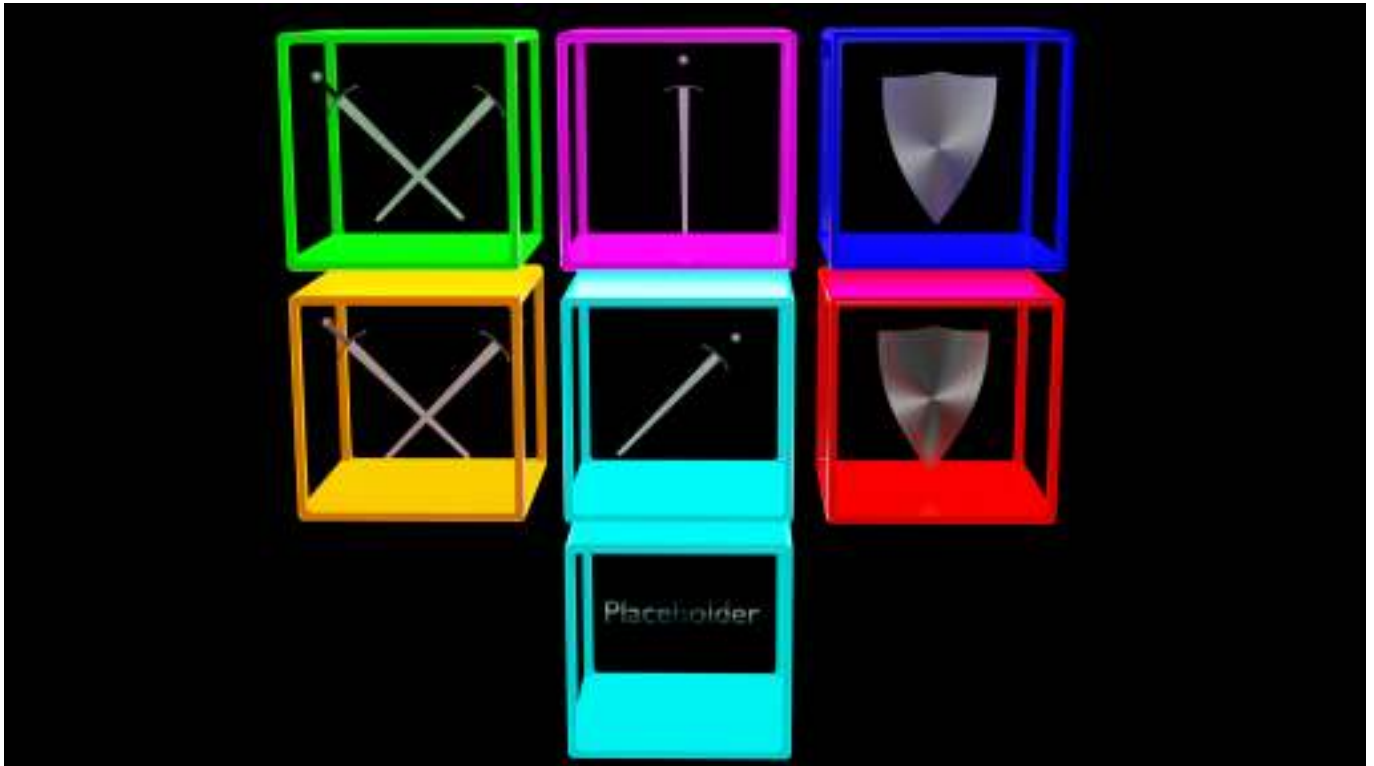


Abbildung A.5.: Ein 3D Mockup des der Beatbuttons des Rhythmuspiels - Teil 2

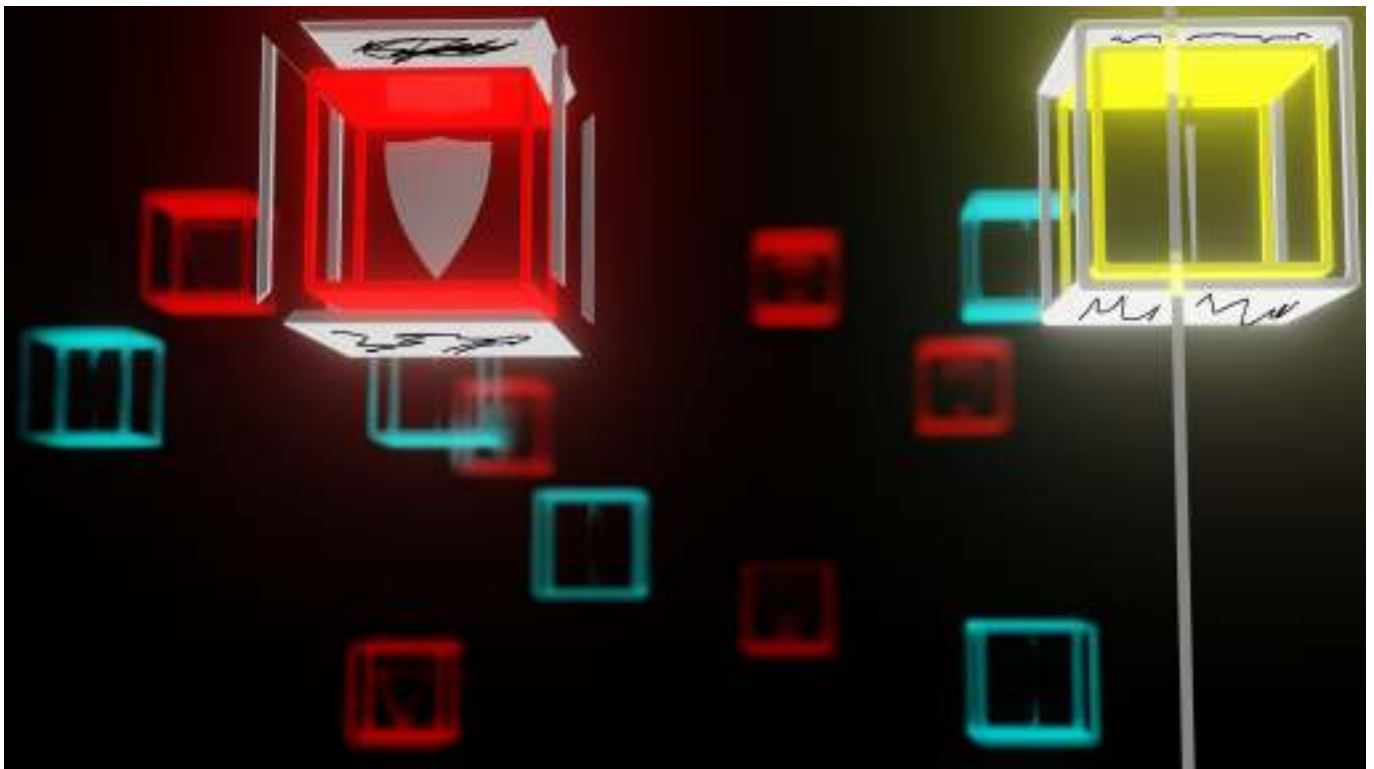


Abbildung A.6.: Ein 3D Mockup des Rhythmuspiels - Teil 1

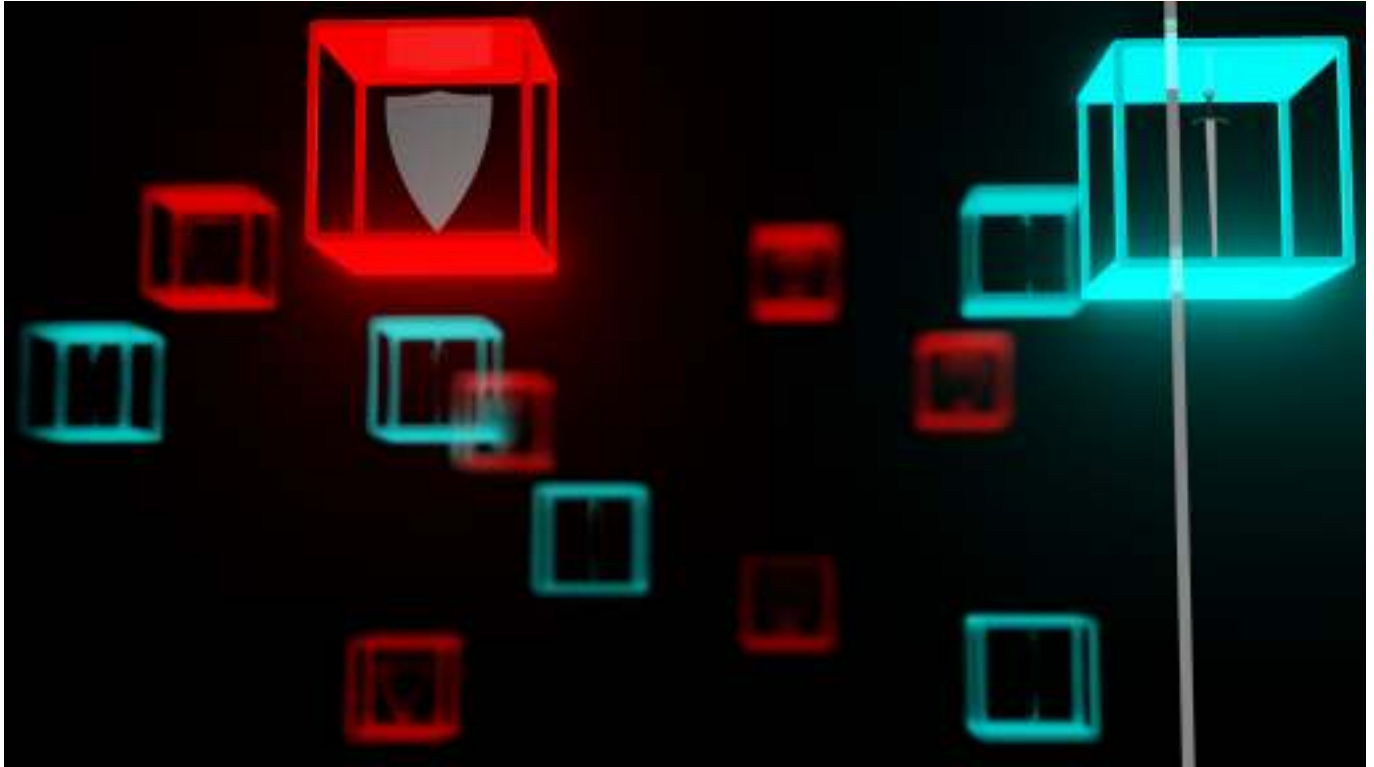


Abbildung A.7.: Ein 3D Mockup des Rhythmusspiels - Teil 2



Abbildung A.8.: Mockups für die Progressbars

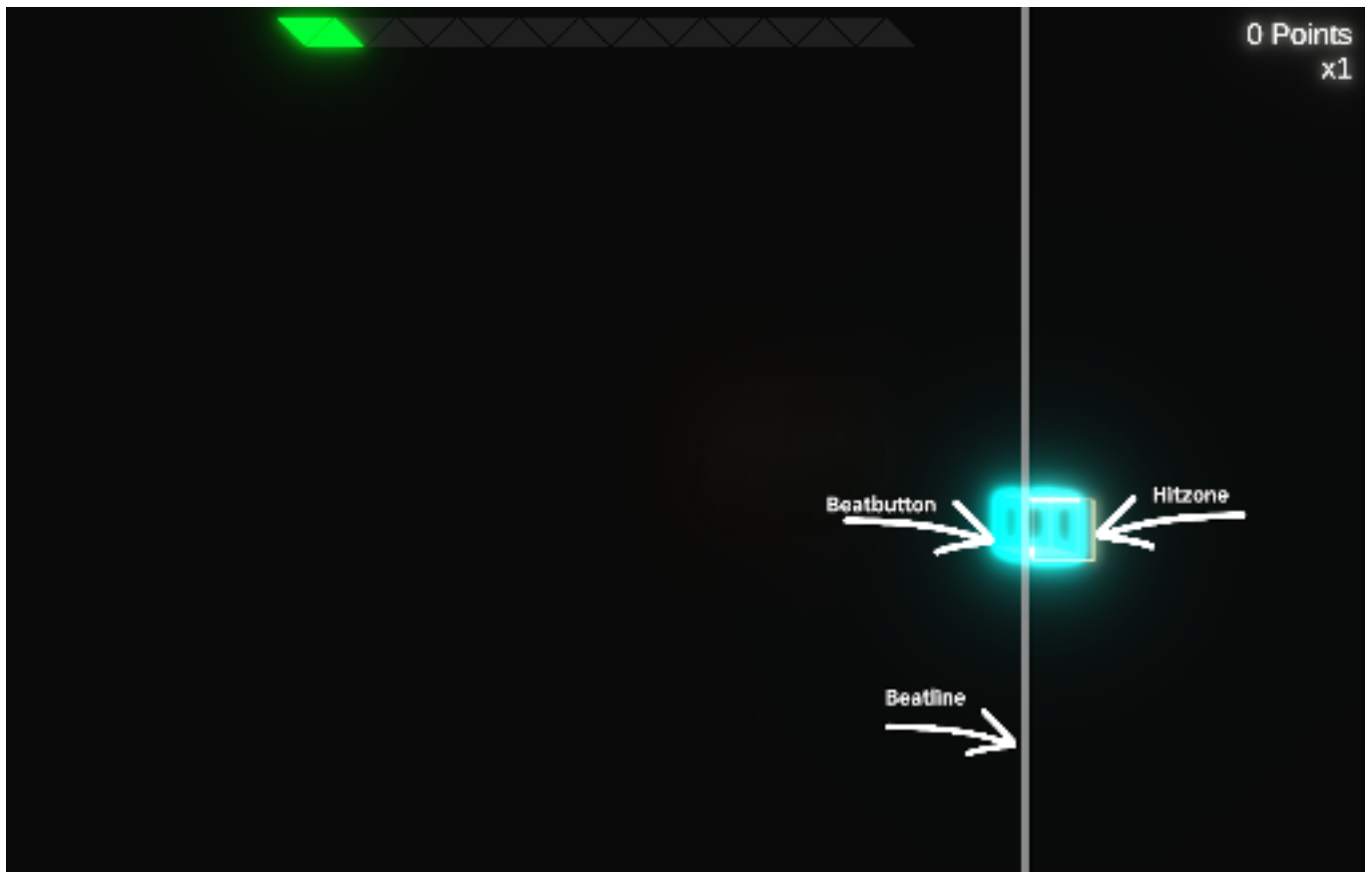


Abbildung A.9.: Veranschaulichung:

Die Beatline bewegt sich horizontal von links nach rechts und würde den Beatbuttons genau in dem Moment schneiden, wenn dieser die Hitzone erreichen würde



Abbildung A.10.: Ein 3D Mockup der Metaspielumgebung



Abbildung A.11.: Raid Shadow Legends Gameplay

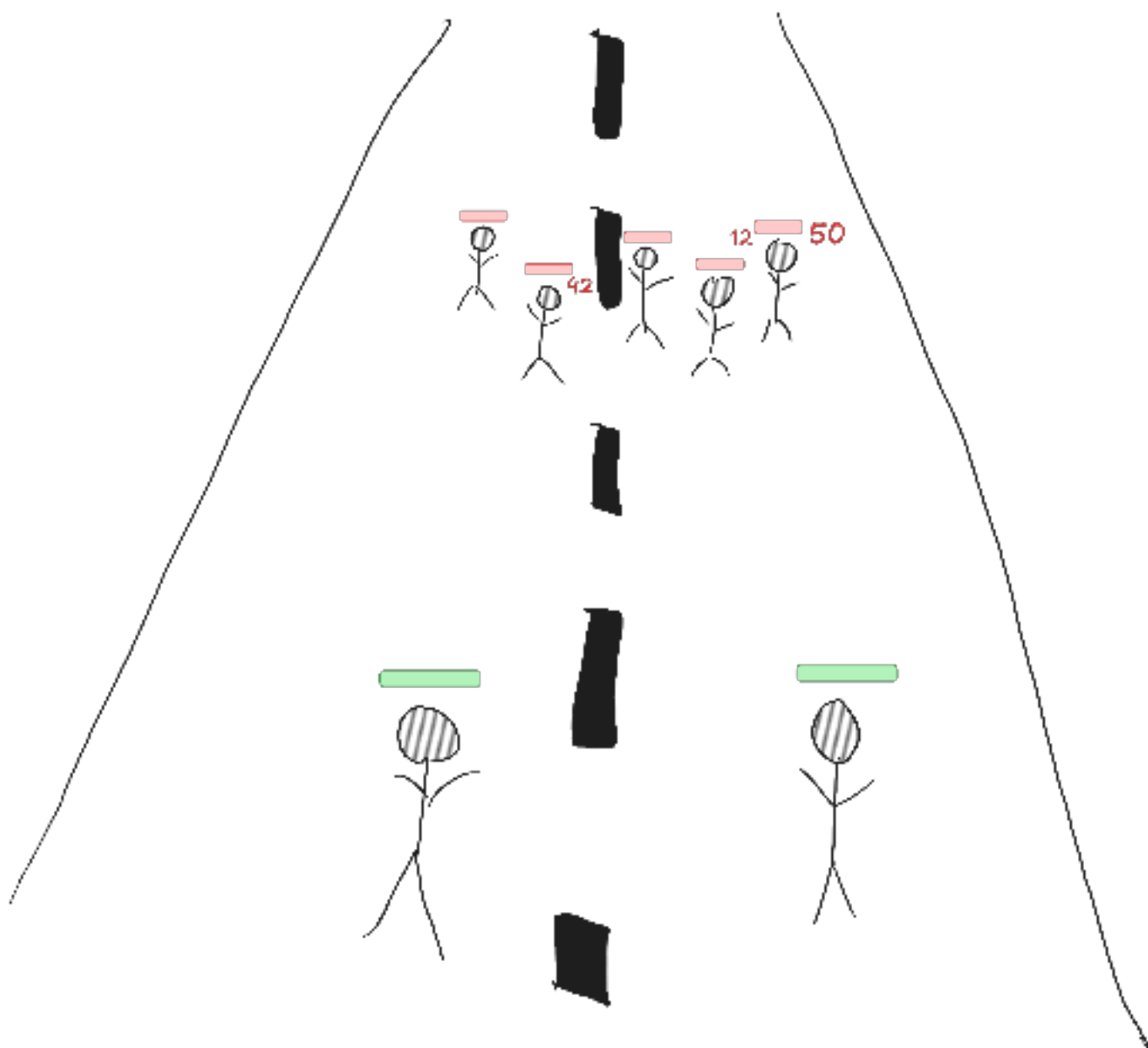


Abbildung A.12.: Metaspiel Variante mit einzelnen Lebensbalken

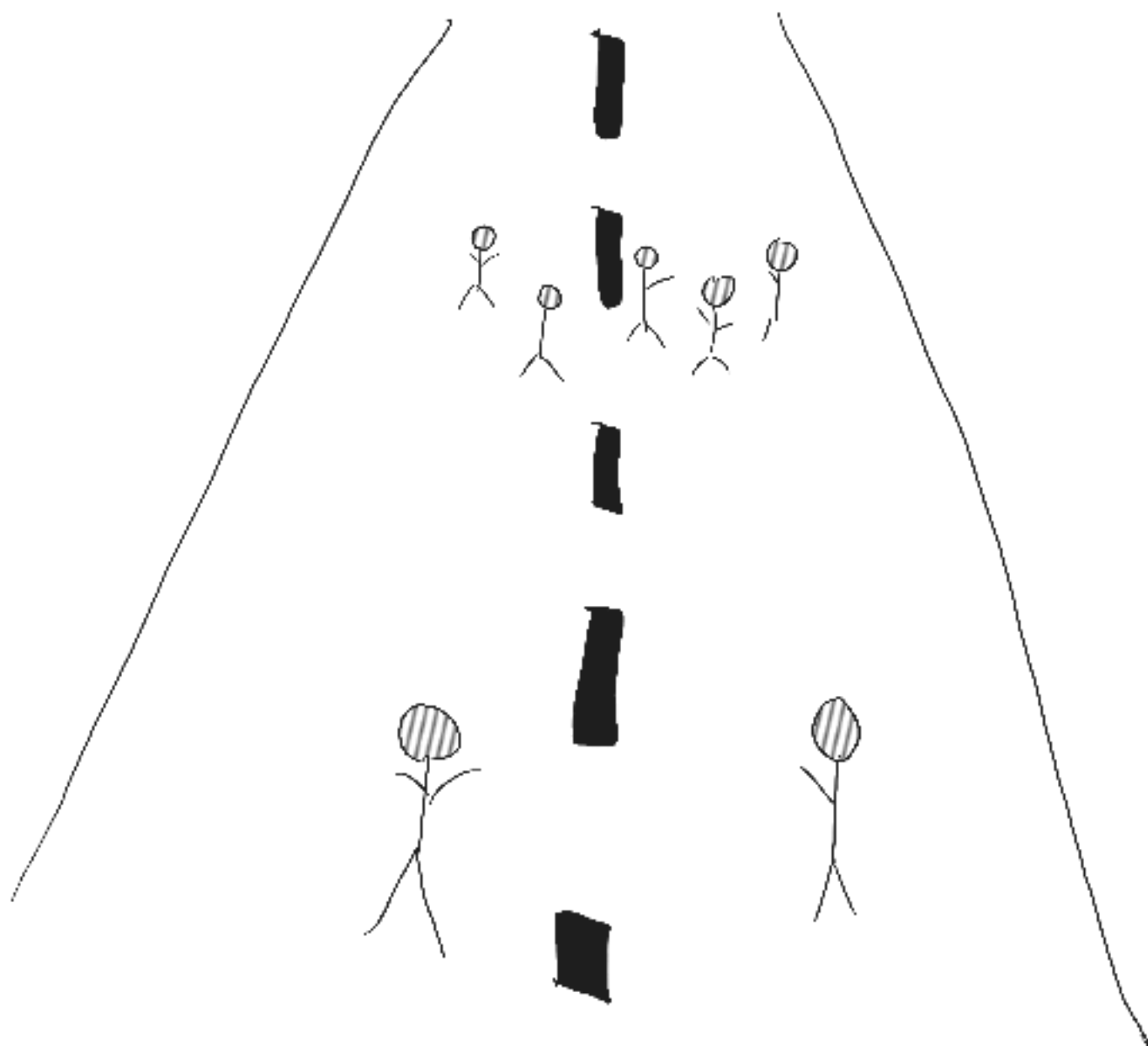


Abbildung A.13.: Metaspiel Variante mit fehlenden Körperteilen als Schadensanzeige

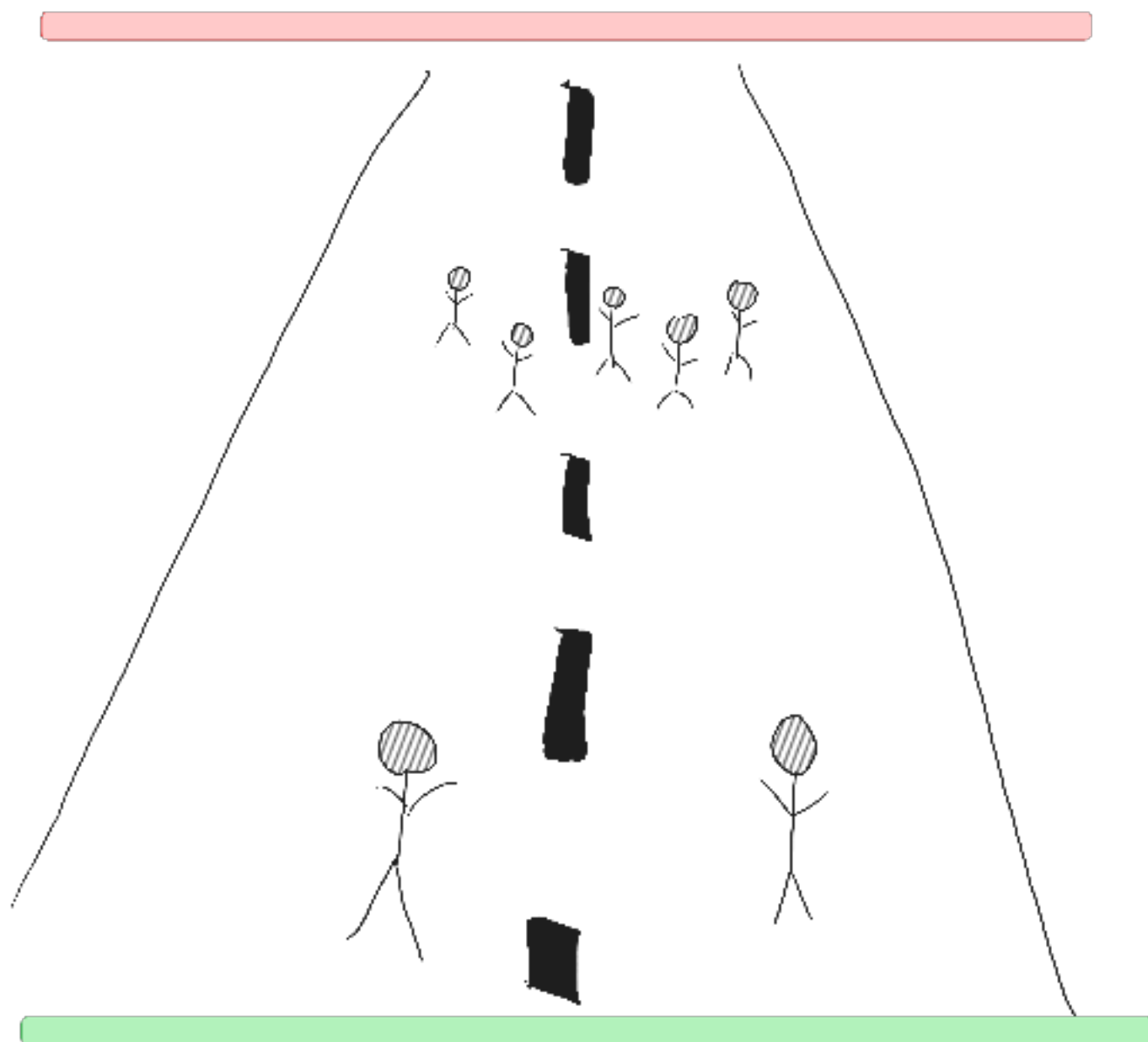


Abbildung A.14.: Metaspiel Variante mit großen Lebensbalken für Gruppen

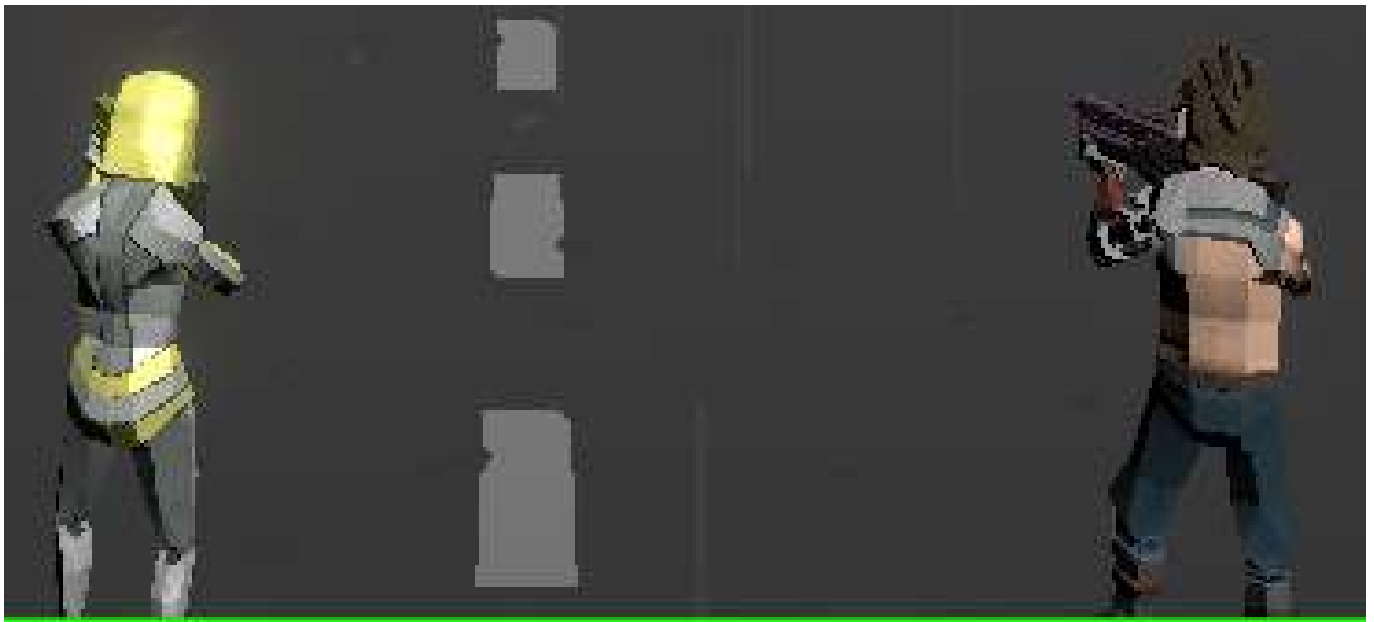


Abbildung A.15.: Angepasste Charatere bevor ein Level ausgewählt wurde



Abbildung A.16.: All möglichen Varianten von Gebäudeteilen



Abbildung A.17.: Objekte die den Gebäuden der Stadt hinzugefügt werden können



Abbildung A.18.: Alle Varianten der fliegenden Autos



Abbildung A.19.: Alle Materialien exemplarisch an einem Gebäudeteil



Abbildung A.20.: Alle verschiedenen Charakter

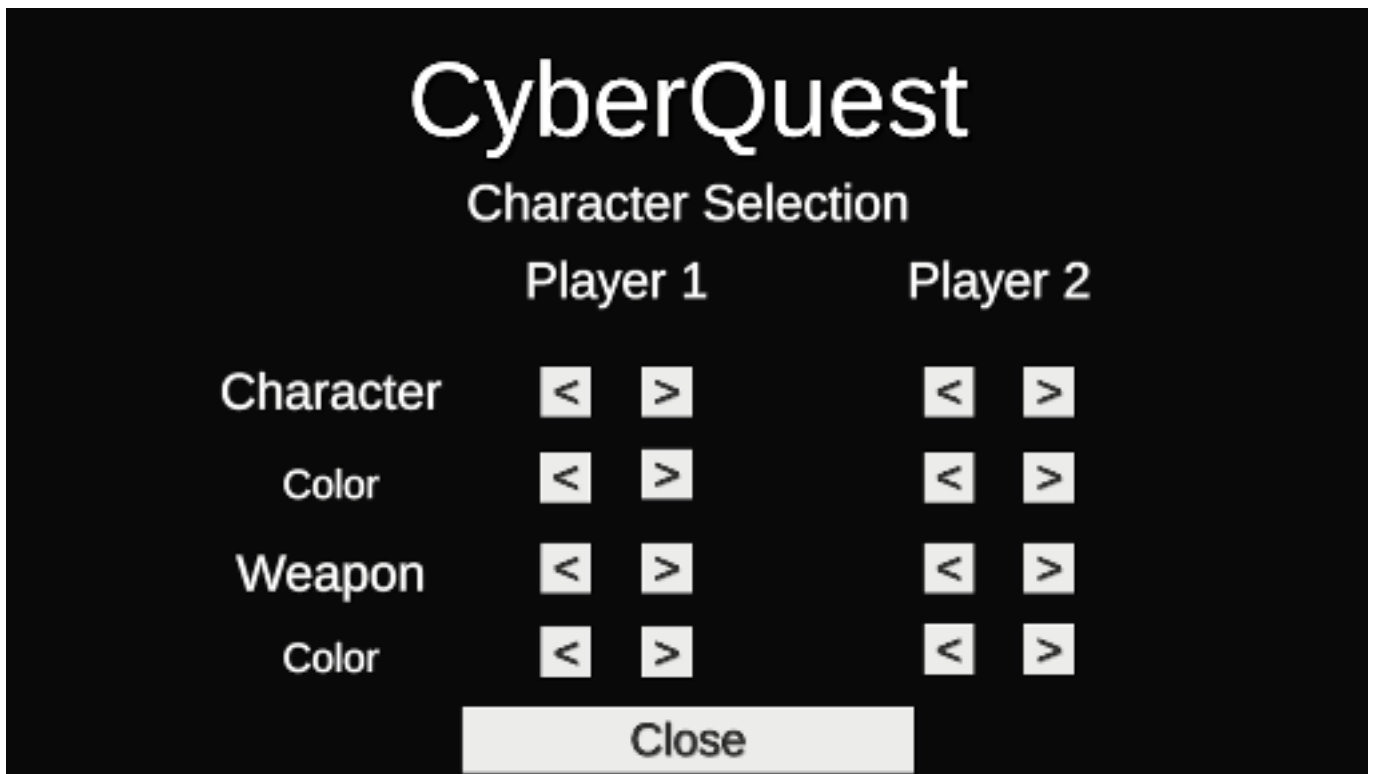


Abbildung A.21.: Das Menü für die Characteranpassung des Metaspiels

CyberQuest

Level Selection

- tsdt
- tsdtgrf
- hghg
- qwqwqw
- sdfrrf
- xd

Close

Abbildung A.22.: Das Menü für die Level-Selektion

#	Exercise	Lead	AB	AC	PF	RG	Summe
0	Management and Coordination	Everyone	56,7h	121,5h	91,8h	50,4h	320,4h
0,1	Planning/Meetings		24,3h	27h	29,7h	21,6h	102,6h
0,2	Documentation		27h	40,5h	54h	21,6h	143,1h
0,3	Presentation/Video		5,4h	54h	8,1h	7,2h	74,7h
1	Exercube Environment System	AB	207,9h	24,3h	0h	9h	241,2h
1,1	Calibration		40,5h	2,7h	0h	0h	43,2h
1,2	Motion Detection		153,9h	21,6h	0h	9h	184,5h
1,3	Display Controller		13,5h	0h	0h	0h	13,5h
1,4	Shader		0h	0h	0h	0h	0h
2	Player Interface	AC	5,4h	21,6h	178,2h	25,2h	230,4h
2,1	Menu		2,7h	8,1h	27h	7,2h	45h
2,2	Settings		2,7h	0h	21,6h	0h	24,3h
2,3	Audio and Visual Feedback		0h	0h	56,7h	9h	65,7h
2,4	Beatmap visualization		0h	0h	72,9h	9h	81,9h
2,5	Game Mode Selection		0h	13,5h	0h	0h	13,5h
2,6	Leaderboard		0h	0h	0h	0h	0h
3	Conductor/Rhythm Game Engine	PF	0h	94,5h	0h	21,6h	116,1h
3,1	Scoring		0h	0h	0h	9h	9h
3,2	Timing Evaluation		0h	0h	0h	5,4h	5,4h
3,3	Timing Manager		0h	0h	0h	7,2h	7,2h
3,4	Beatmap Creator		0h	94,5h	0h	0h	94,5h
3,5	Coop Manager		0h	0h	0h	0h	0h
4	Metagame Integration System	RG	0h	8,1h	0h	73,8h	81,9h
4,1	Autobattler Engine		0h	0h	0h	19,8h	19,8h
4,2	Performance Evaluation		0h	0h	0h	10,8h	10,8h
4,3	Player		0h	0h	0h	3,6h	3,6h
4,4	Classes		0h	0h	0h	0h	0h
4,5	Game State		0h	8,1h	0h	5,4h	13,5h
4,6	Enemy Controller		0h	0h	0h	5,4h	5,4h
4,7	Level Design		0h	0h	0h	18h	18h
4,8	Level Progress Bar		0h	0h	0h	10,8h	10,8h
5	Tutorial	AB	0h	0h	0h	0h	0h
Total expenditure in hours			270	270	270	180	990h

Abbildung A.23.: Der finale Projektplan



A.6. Protokolle













































AB	Artjom Becker	8
AC	Ali Cebeci	8
PF	Philip Fauss	8
RG	Ruben Gielisch	8
GEQ	Game Experience Questionnaire	4